ARIMA - Modelo Auto-Regressivo Integrado com Médias Móveis

PNV-3421 – Processos Estocásticos

Prof. Dr. André Bergsten Mendes

Bibliografia Principal

- Hanke, J.E & Reitsch A.G. (1998) Business Forecasting. 6th Edition, Prentice Hall, Upper Sadle River, NJ.

Modelo Autorregressivo ("AR")

- Um modelo autorregressivo de ***p*** períodos tem a seguinte estrutura:

$$\widehat{Y_t} = \phi_0 + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p}$$

$$\epsilon_t = Y_t - \widehat{Y_t}$$

Onde:

- $\widehat{Y_t}$ - variável dependente
- $Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p}$ - variáveis independentes (série histórica com defasagem de tempo)
- $\phi_1, \phi_2, \ldots, \phi_p$ - coeficientes de regressão
- ϵ_t - resíduo, que representa eventos aleatórios não explicados pelo modelo

Modelo Autorregressivo ("AR")

- Os coeficientes $\phi_1, \phi_2, \ldots, \phi_p$ são obtidos por meio do método dos mínimos quadrados *não-linear*, aplicando um algoritmo iterativo.
- A variância da expressão de Y_t leva em consideração que as variáveis 'independentes' são correlacionadas.
- Para realizar a previsão:

$$\widehat{Y_t} = \phi_0 + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p}$$

Modelo Autorregressivo ("AR")

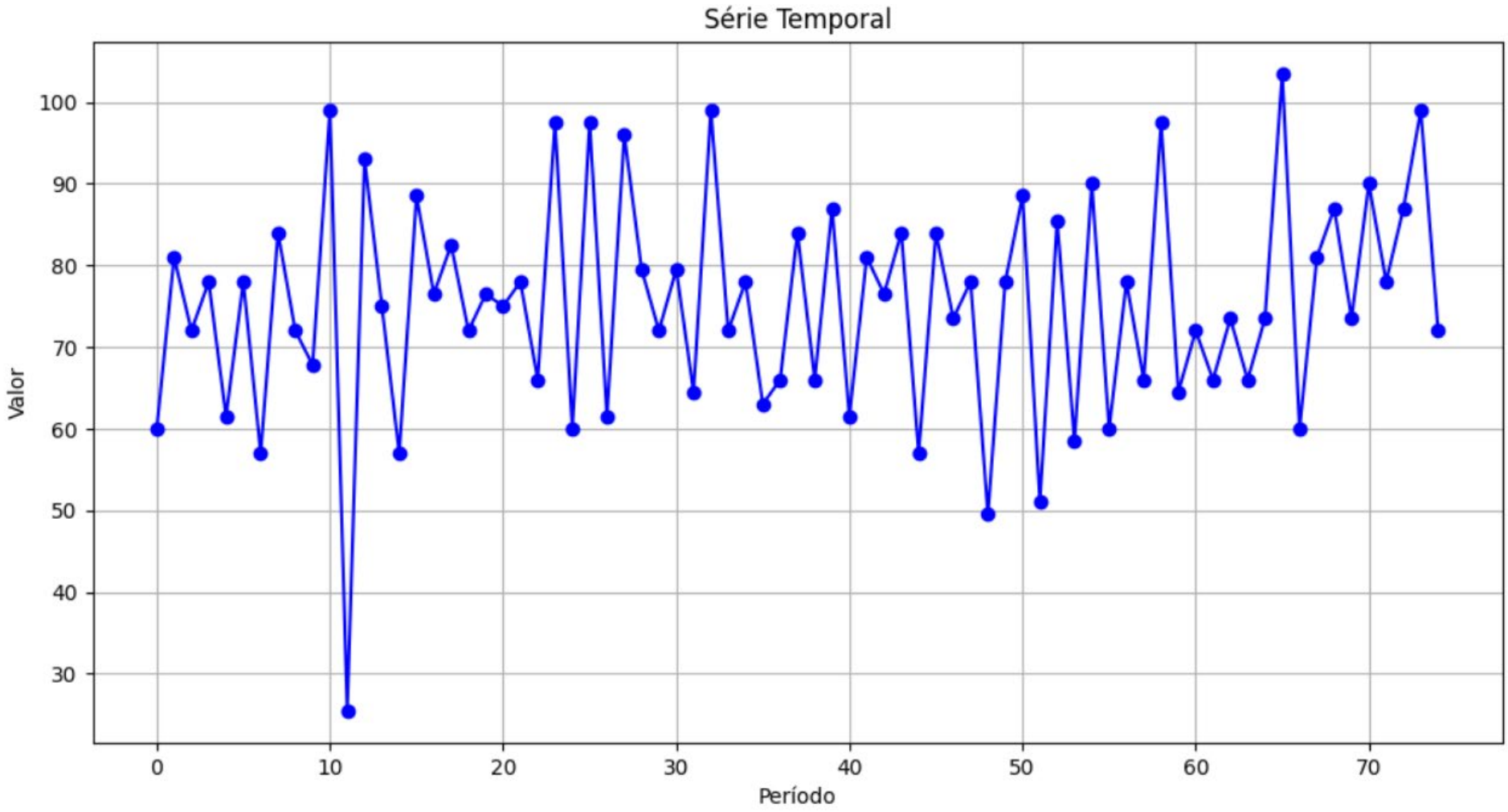
- Exemplo:

60,0	88,5	79,5	84,0	72,0
81,0	76,5	64,5	73,5	66,0
72,0	82,5	99,0	78,0	73,5
78,0	72,0	72,0	49,5	66,0
61,5	76,5	78,0	78,0	73,5
78,0	75,0	63,0	88,5	103,5
57,0	78,0	66,0	51,0	60,0
84,0	66,0	84,0	85,5	81,0
72,0	97,5	66,0	58,5	87,0
67,8	60,0	87,0	90,0	73,5
99,0	97,5	61,5	60,0	90,0
25,5	61,5	81,0	78,0	78,0
93,0	96,0	76,5	66,0	87,0
75,0	79,5	84,0	97,5	99,0
57,0	72,0	57,0	64,5	72,0

```python
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.ar_model import AutoReg
```

```python
# Dados da série temporal
series = np.array([
    60.0, 81.0, 72.0, 78.0, 61.5, 78.0, 57.0, 84.0, 72.0, 67.8, 99.0, 25.5,
    93.0, 75.0, 57.0, 88.5, 76.5, 82.5, 72.0, 76.5, 75.0, 78.0, 66.0, 97.5,
    60.0, 97.5, 61.5, 96.0, 79.5, 72.0, 79.5, 64.5, 99.0, 72.0, 78.0, 63.0,
    66.0, 84.0, 66.0, 87.0, 61.5, 81.0, 76.5, 84.0, 57.0, 84.0, 73.5, 78.0,
    49.5, 78.0, 88.5, 51.0, 85.5, 58.5, 90.0, 60.0, 78.0, 66.0, 97.5, 64.5,
    72.0, 66.0, 73.5, 66.0, 73.5, 103.5, 60.0, 81.0, 87.0, 73.5, 90.0, 78.0,
    87.0, 99.0, 72.0
])
# Plot da série temporal
plt.figure(figsize=(12, 6))
plt.plot(series, marker='o', linestyle='-', color='b')
plt.title("Série Temporal")
plt.xlabel("Período")
plt.ylabel("Valor")
plt.grid(True)
plt.show()
```

Modelo Autorregressivo (“AR”)



Modelo Autorregressivo (“AR”)

```
# Ajuste dos modelos AR(2) e AR(3)
model_ar2 = AutoReg(series, lags=2).fit()
model_ar3 = AutoReg(series, lags=3).fit()

# Parâmetros dos modelos
params_ar2 = model_ar2.params
params_ar3 = model_ar3.params

# Exibir os parâmetros
print("Parâmetros do modelo AR(2):")
print(params_ar2)

print("\nParâmetros do modelo AR(3):")
print(params_ar3)
```

✓ 0.0s

```
Parâmetros do modelo AR(2):
[ 1.14935315e+02 -5.28880827e-01  4.90523095e-03]

Parâmetros do modelo AR(3):
[93.30087168 -0.52552808  0.11005637  0.18026002]
```

Modelo Autorregressivo (“AR”)

```
# Previsões dado a dado (valores ajustados) e previsão para o próximo período
fitted_values_ar2 = model_ar2.predict(start=2, end=len(series) - 1)
fitted_values_ar3 = model_ar3.predict(start=3, end=len(series) - 1)
next_prediction_ar2 = model_ar2.predict(start=len(series), end=len(series))
next_prediction_ar3 = model_ar3.predict(start=len(series), end=len(series))

# Visualização
plt.figure(figsize=(14, 6))

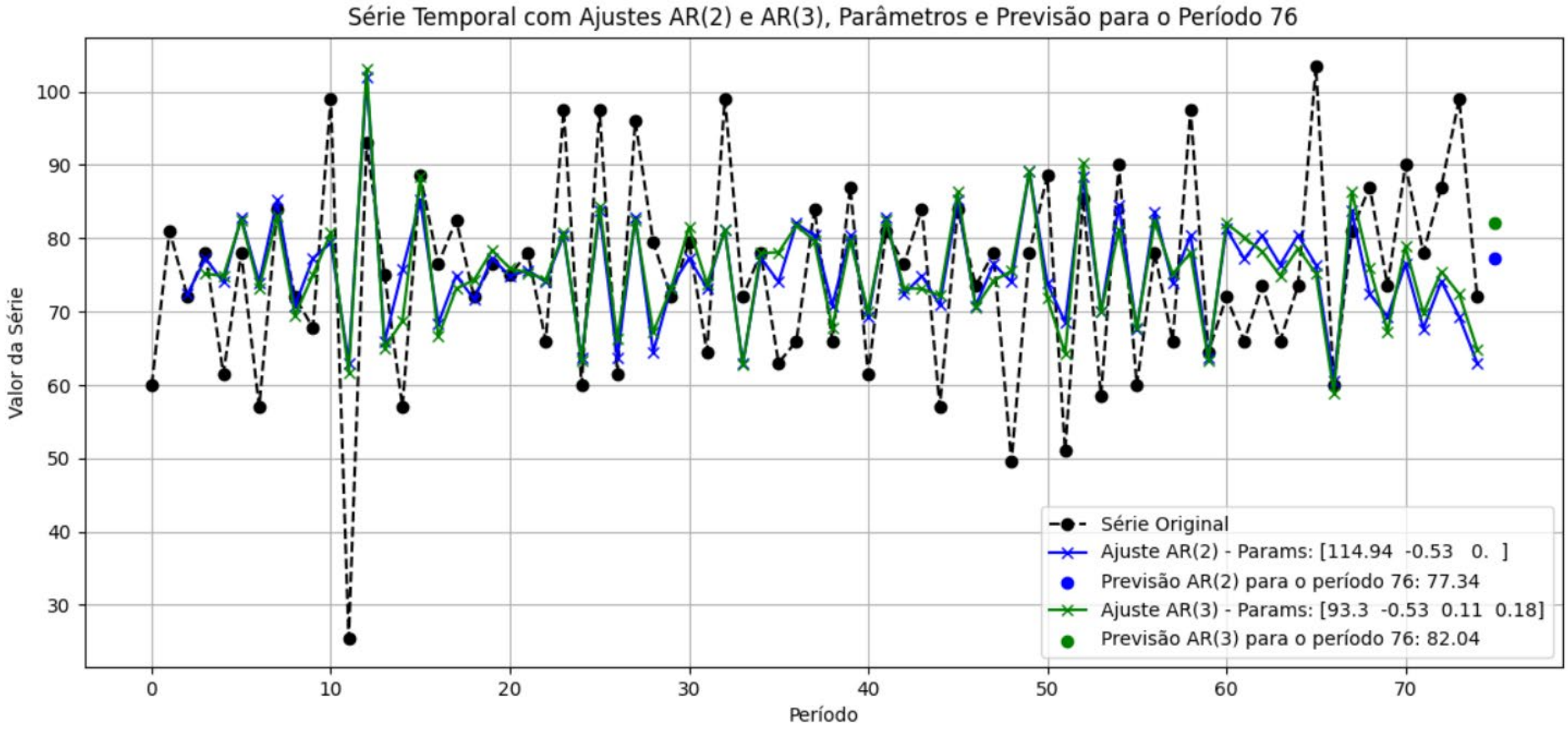
# Série temporal original
plt.plot(series, label="Série Original", color="black", marker="o", linestyle="dashed")

# Valores ajustados para AR(2) e previsão para o próximo período
plt.plot(range(2, len(series)), fitted_values_ar2, label=f"Ajuste AR(2) - Params: {np.round(params_ar2, 2)}", color="blue", marker="x")
plt.scatter(len(series), next_prediction_ar2, color="blue", label=f"Previsão AR(2) para o período 76: {next_prediction_ar2[0]:.2f}")

# Valores ajustados para AR(3) e previsão para o próximo período
plt.plot(range(3, len(series)), fitted_values_ar3, label=f"Ajuste AR(3) - Params: {np.round(params_ar3, 2)}", color="green", marker="x")
plt.scatter(len(series), next_prediction_ar3, color="green", label=f"Previsão AR(3) para o período 76: {next_prediction_ar3[0]:.2f}")

# Configurações de layout
plt.xlabel("Período")
plt.ylabel("Valor da Série")
plt.title("Série Temporal com Ajustes AR(2) e AR(3), Parâmetros e Previsão para o Período 76")
plt.legend()
plt.grid()
plt.show()
```

Modelo Autorregressivo (“AR”)



Modelo Autorregressivo (“AR”)

```
# Cálculo do Mean Squared Error (MSE) entre os valores ajustados e os valores reais da série
mse_ar2 = mean_squared_error(series[2:], fitted_values_ar2)  # Ignorando os dois primeiros valores devido à defasagem

# Cálculo do Mean Squared Error (MSE) entre os valores ajustados e os valores reais da série
mse_ar3 = mean_squared_error(series[3:], fitted_values_ar3)  # Ignorando os dois primeiros valores devido à defasagem

print(mse_ar2,mse_ar3)
```

```
136.14209368558102 133.9772400401187
```

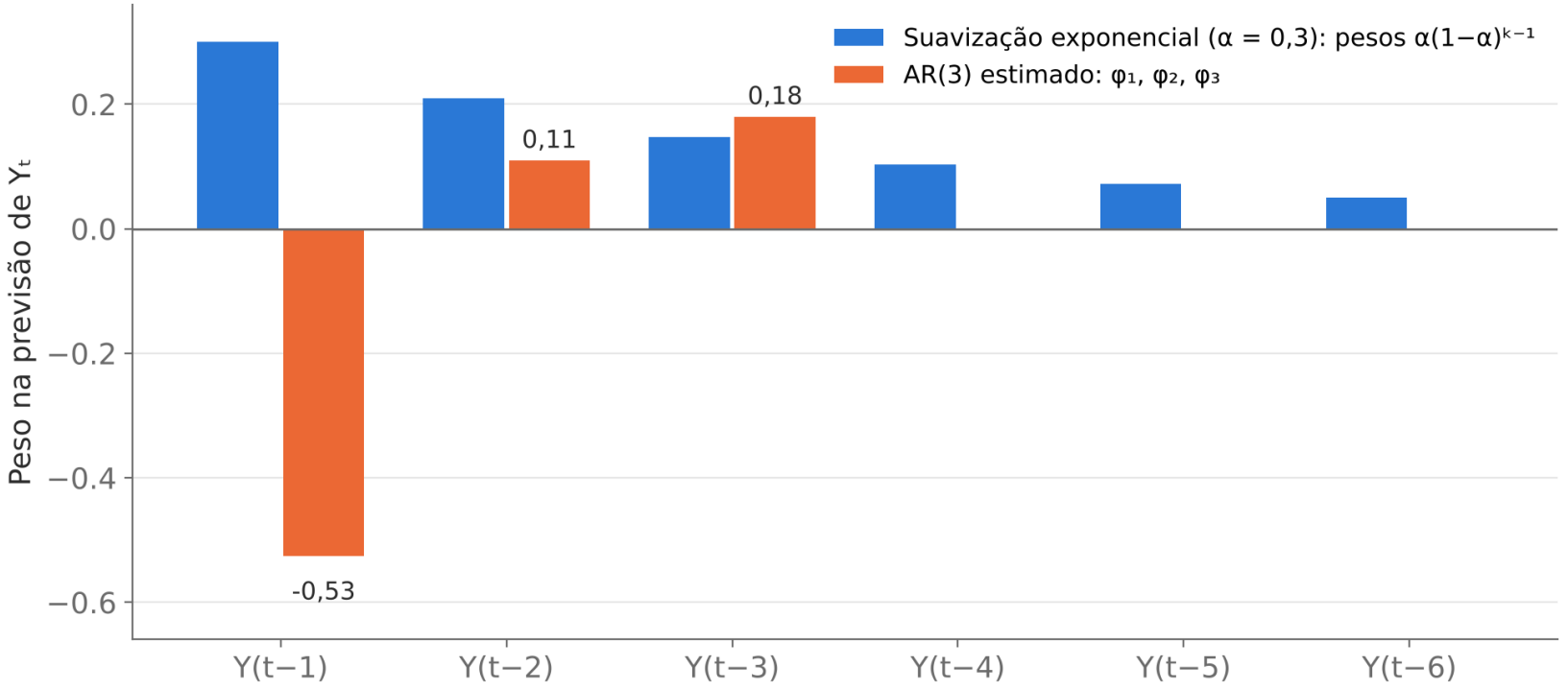
Suavização Exponencial × AR

- Suavização exponencial simples: $\hat{Y}_{t+1} = \alpha Y_t + \alpha(1-\alpha)\hat{Y}_t \quad 0 < \alpha \leq 1$
- Expandindo a recursão, a previsão é uma média ponderada de todo o histórico:

$$\hat{Y}_{t+1} = \alpha Y_t + \alpha(1-\alpha)Y_{t-1} + \alpha(1-\alpha)^2 Y_{t-2} + \cdots$$

- Tem a forma de um AR de ordem infinita, mas com pesos **fixos**: todos positivos, decrescendo exponencialmente, somando 1 e controlados por um único parâmetro α.
- No AR(p) os pesos $\phi_1, \ldots, \phi_p$ são **livres**: estimados por mínimos quadrados, podem ser negativos, e a constante ϕ_0 define uma média de longo prazo.
- Consequência: a suavização exponencial supõe que o próximo valor se parece com os recentes e que os choques são permanentes. Por isso, é adequada a séries cujo nível local muda ao longo do tempo, sem um patamar de equilíbrio. O AR, por ter pesos livres, captura correlação negativa, isto é, séries em zigue-zague ($\varphi_1 < 0$) e séries estacionárias, em que os choques são temporários e a previsão retorna à média μ.

Suavização Exponencial × AR (Exemplo)



- Na série do slide 5, o AR(3) estima $\phi_1 = -0{,}53$, um valor alto tende a ser seguido por um valor baixo.
- Nenhum α da suavização exponencial produz um peso negativo, então ela não consegue explorar esse padrão (ver comparação após o modelo MA).

Modelo de Média Móvel ("MA")

- Um modelo de média móvel de ***q*** períodos tem a seguinte estrutura:

$$\widehat{Y}_t = w_0 + w_1\epsilon_{t-1} + w_2\epsilon_{t-2} + \cdots + w_q\epsilon_{t-q}$$

$$\epsilon_t = Y_t - \widehat{Y}_t$$

Onde:

- $\widehat{Y}_t$- variável dependente
- $\epsilon_{t-1}, \epsilon_{t-2}, \ldots, \epsilon_{t-q}$ - resíduos de períodos anteriores (variáveis independentes), $w_1, w_2, \ldots, w_q$ - pesos
- "MA" – *Moving average*
- Para realizar a previsão:

$$\widehat{Y}_t = w_0 + w_1\epsilon_{t-1} + w_2\epsilon_{t-2} + \cdots + w_q\epsilon_{t-q}$$

Modelo de Média Móvel (“MA”)

- Exemplo (mesmo que o anterior):

60,0	88,5	79,5	84,0	72,0
81,0	76,5	64,5	73,5	66,0
72,0	82,5	99,0	78,0	73,5
78,0	72,0	72,0	49,5	66,0
61,5	76,5	78,0	78,0	73,5
78,0	75,0	63,0	88,5	103,5
57,0	78,0	66,0	51,0	60,0
84,0	66,0	84,0	85,5	81,0
72,0	97,5	66,0	58,5	87,0
67,8	60,0	87,0	90,0	73,5
99,0	97,5	61,5	60,0	90,0
25,5	61,5	81,0	78,0	78,0
93,0	96,0	76,5	66,0	87,0
75,0	79,5	84,0	97,5	99,0
57,0	72,0	57,0	64,5	72,0

Modelo de Média Móvel (“MA”)

```python
from statsmodels.tsa.arima.model import ARIMA

# Ajuste do modelo MA(1) - apenas o termo de média móvel
model_ma1 = ARIMA(series, order=(0, 0, 1)).fit()

# Previsão para a série e próximo período
fitted_values_ma1 = model_ma1.fittedvalues
next_prediction_ma1 = model_ma1.forecast(steps=1)

# Exibir os parâmetros do modelo MA(1)
params_ma1 = model_ma1.params
print("Parâmetros do modelo MA(1):")
print(params_ma1)
```

```
Parâmetros do modelo MA(1):
[ 75.27105719  -0.398386   148.44658216]
```

Modelo de Média Móvel (“MA”)

```
# Previsão para toda a série e próximo período
fitted_values_ma1 = model_ma1.fittedvalues
forecast_values = model_ma1.get_forecast(steps=10)  # Previsão para 10 períodos à frente
forecast_index = np.arange(len(series), len(series) + 10)

# Obter intervalo de confiança
forecast_ci = forecast_values.conf_int()

# Visualização da série temporal e previsão
plt.figure(figsize=(14, 6))

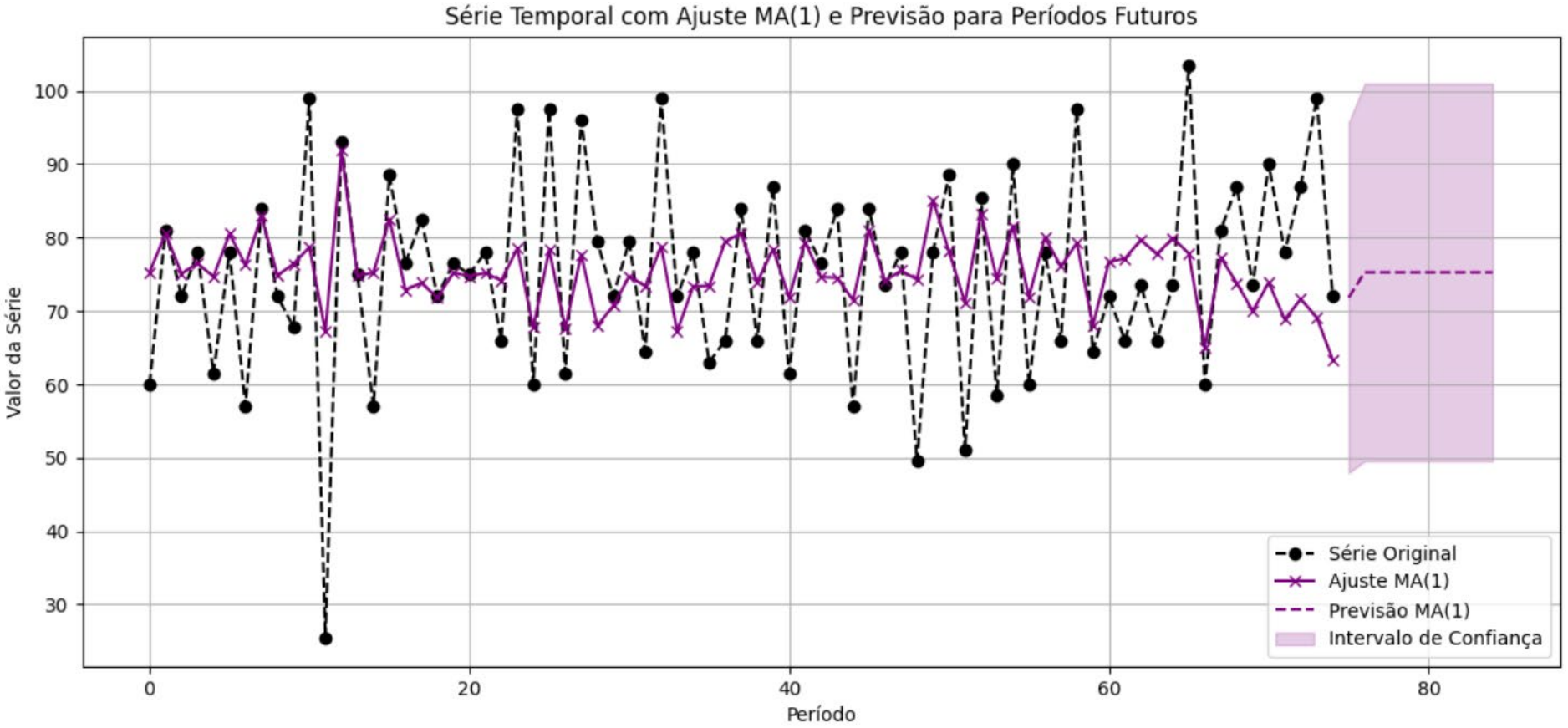
# Série temporal original
plt.plot(series, label="Série Original", color="black", marker="o", linestyle="dashed")

# Valores ajustados para MA(1)
plt.plot(fitted_values_ma1, label="Ajuste MA(1)", color="purple", marker="x")

# Previsão para os próximos períodos
plt.plot(forecast_index, forecast_values.predicted_mean, color="purple", linestyle="--", label="Previsão MA(1)")
plt.fill_between(forecast_index, forecast_ci[:, 0], forecast_ci[:, 1], color="purple", alpha=0.2, label="Intervalo de Confiança")

# Configurações de layout
plt.xlabel("Período")
plt.ylabel("Valor da Série")
plt.title("Série Temporal com Ajuste MA(1) e Previsão para Períodos Futuros")
plt.legend()
plt.grid()
plt.show()
```

Modelo de Média Móvel (“MA”)



Modelo de Média Móvel (“MA”)

```
from sklearn.metrics import mean_squared_error

# Cálculo do Mean Squared Error (MSE) entre os valores ajustados e os valores reais da série
mse_ma1 = mean_squared_error(series[1:], fitted_values_ma1[1:])  # Ignorando o primeiro valor por falta de previsão

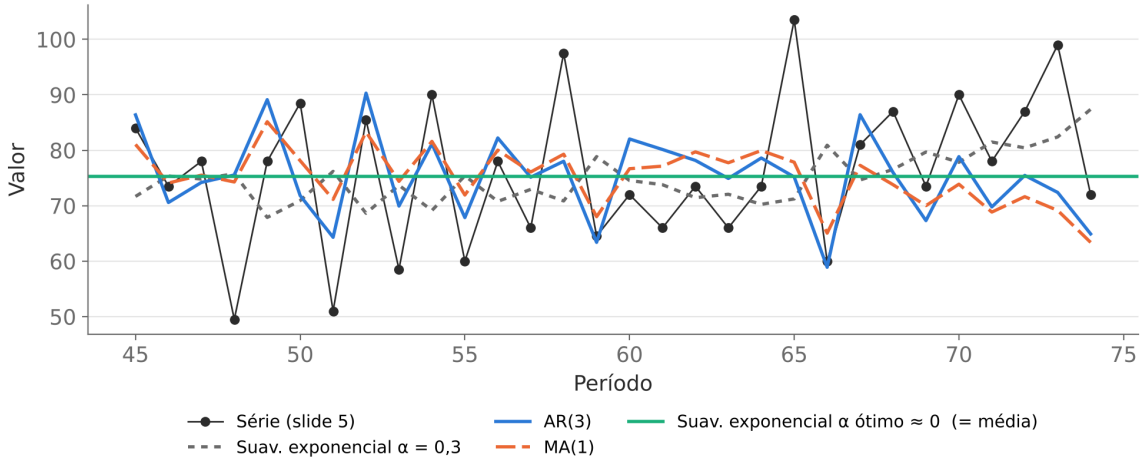
mse_ma1
```

```
np.float64(147.73378670012744)
```

Suavização Exponencial × MA

- A suavização exponencial pode ser escrita como correção pelo erro:
- $\hat{Y}_{t+1} = S_t = \alpha Y_t + (1-\alpha)\hat{Y}_t = \hat{Y}_t + \alpha\left(Y_t - \hat{Y}_t\right) = \hat{Y}_t + \alpha\varepsilon_t \Rightarrow \varepsilon_t = Y_t - \hat{Y}_t \Rightarrow \hat{Y}_t = Y_t - \varepsilon_t$
- Substituindo: $\hat{Y}_{t+1} = \hat{Y}_t + \alpha\varepsilon_t \Rightarrow Y_{t+1} - Y_t = \varepsilon_{t+1} - (1-\alpha)\varepsilon_t$
- Ou seja, a suavização exponencial é um **ARIMA(0,1,1)**: um MA(1) aplicado à série **diferenciada**, com coeficiente $\theta = -(1-\alpha)$, sempre negativo ou nulo.
- O MA(q) dos slides anteriores atua sobre a série original (d = 0): após q períodos a previsão volta à média w_0. A suavização exponencial não tem média fixa: a previsão fica no último nível estimado.
- Verificação (série do slide 5): o ARIMA(0,1,1) estima $\theta = -0{,}997$, logo $\alpha = 1 + \theta = 0{,}003$, o mesmo α ≈ 0 obtido ajustando a suavização exponencial diretamente.

Comparação: Suavização Exponencial, AR e MA



Modelo	MSE
Suav. exp., α ótimo ≈ 0	187,7
Suav. exp., α = 0,3	246,9
MA(1)	147,7
AR(2)	136,1
AR(3)	**134,0**

- Com α ótimo ≈ 0, a suavização exponencial ignora o histórico e prevê a média (75,2).
- AR e MA usam a correlação negativa entre períodos vizinhos e reduzem o erro em ~25%.
- MSE dentro da amostra, mesmo critério dos slides 11 e 17.

Modelo Autoregressivo com Média Móvel ("ARMA")

- Um modelo autoregressivo de ***p*** períodos de defasagem das variáveis 'independentes' Y_t, e ***q*** períodos de defasagem dos erros ϵ_t tem a seguinte estrutura:

$$\widehat{Y_t} = \theta_0 + \sum_{i=1}^{p} \phi_i Y_{t-i} + \sum_{i=1}^{q} w_i \epsilon_{t-i}$$

$$\epsilon_t = Y_t - \widehat{Y_t}$$

- "ARMA" – *Autoregressive moving average model*
- Usa-se a notação ARMA(p,q).

Série Estacionária

- Os modelos do tipo ARMA(p,q) só são aplicáveis a séries estacionárias Y_t, em que as seguintes características são observadas:
  1. A média $\mathrm{E}(Y_t)$ é a mesma para todo instante *t*.
  2. A variância de Y_t é a mesma para todo instante *t*.

- Para testar se uma série histórica é ou não estacionária, podemos aplicar o teste de Dickey-Fuller. Teste de hipótese:

$$H_0: não\ estiacionária$$
$$H_1: estacionária$$

Série Estacionária

Teste de Dickey-Fuller

- Considere a série histórica abaixo. Aplique o teste de Dickey-Fuller para verificar se a série é estacionária.

t	yt	t	yt
1	13,4	11	27,8
2	11,8	12	32,2
3	15,3	13	31,1
4	18,7	14	36,7
5	18,9	15	37,7
6	21,1	16	37,7
7	21,9	17	39,1
8	25,6	18	41,4
9	26,4	19	41,8
10	27,6	20	44,2

Série Estacionária

Teste de Dickey-Fuller

```
import numpy as np
import pandas as pd
```

✓ 0.4s

```
data1 = [13.4, 11.8, 15.3, 18.7, 18.9, 21.1, 21.9, 25.6, 26.4, 27.6,
         27.8, 32.2, 31.1, 36.7, 37.7, 37.7, 39.1, 41.4, 41.8, 44.2]

df1 = pd.DataFrame(
    data = {'Yt': data1},
)
df1.head()
```

✓ 0.0s

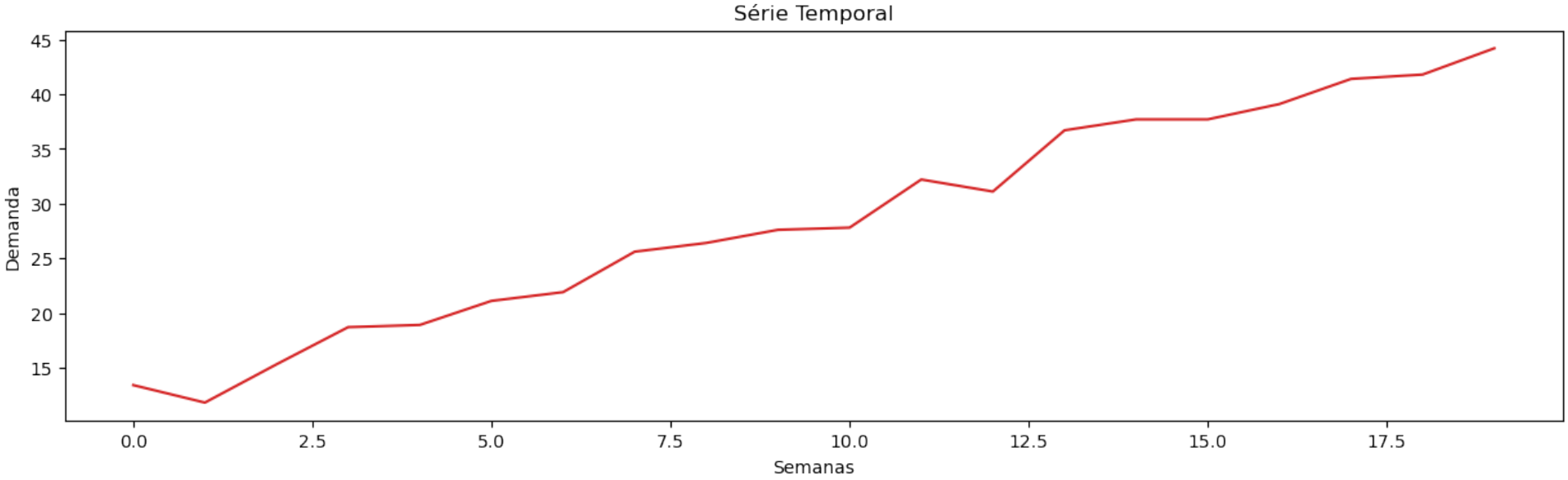
	Yt
0	13.4
1	11.8
2	15.3
3	18.7
4	18.9

Série Estacionária

Teste de Dickey-Fuller

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15,4), dpi=100)
plt.plot(df1.index, df1.Yt, color='tab:red')
plt.gca().set(title='Série Temporal', xlabel='Semanas', ylabel='Demanda')
plt.show()
```



Série Estacionária

Teste de Dickey-Fuller

```
from statsmodels.tsa.stattools import adfuller
from numpy import log

result = adfuller(df1.Yt.dropna())
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
```

```
ADF Statistic: 8.680626
p-value: 1.000000
```

Aceitamos a hipótese nula, e a série é não-estacionária.

Série Estacionária

- Quando uma série não for estacionária, podemos aplicar pelo menos duas técnicas para torná-la estacionária: i) *detrending* - remoção da componente de tendência (*trend*); ii) *differencing* – subtrair o dado da série histórica do período *t-1* do dado período *t*. Se necessário, repetir este processo ***d*** vezes.
- Também é necessário remover a componente de sazonalidade, quando houver.

Série Estacionária

- Exemplo de *differencing* – subtrair o dado da série histórica do período *t-1* do dado período *t*:

t	yt	t	yt
1		11	0,2
2	-1,6	12	4,4
3	3,6	13	-1,0
4	3,4	14	5,6
5	0,2	15	1,0
6	2,2	16	0,0
7	0,8	17	1,4
8	3,8	18	2,4
9	0,8	19	0,4
10	1,2	20	2,4

Série Estacionária

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt
plt.rcParams.update({'figure.figsize':(9,7), 'figure.dpi':120})

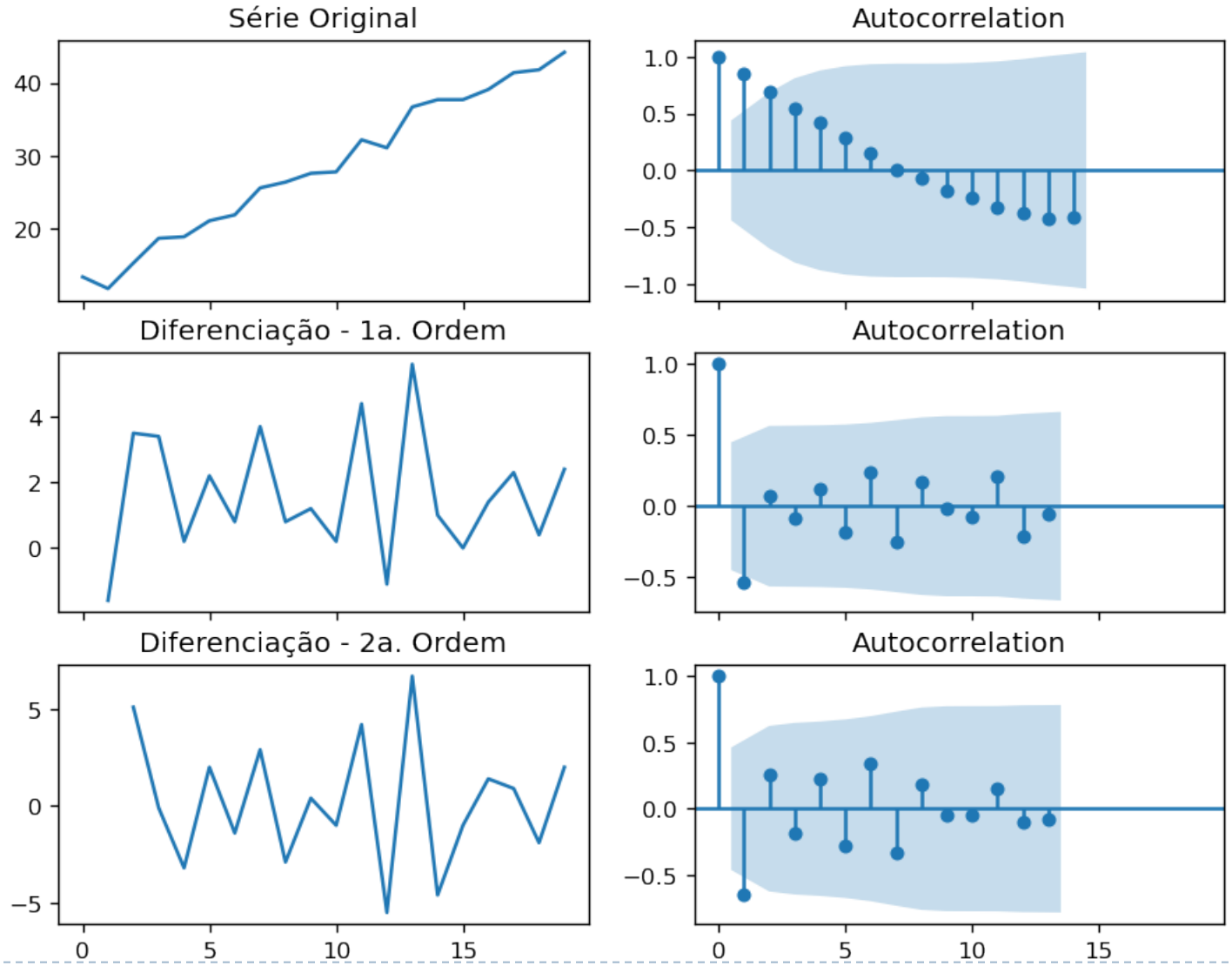

# Original Series
fig, axes = plt.subplots(3, 2, sharex=True)
axes[0, 0].plot(df1.Yt); axes[0, 0].set_title('Série Original')
plot_acf(df1.Yt, ax=axes[0, 1])

# 1st Differencing
axes[1, 0].plot(df1.Yt.diff()); axes[1, 0].set_title('Diferenciação - 1a. Ordem')
plot_acf(df1.Yt.diff().dropna(), ax=axes[1, 1])

# 2nd Differencing
axes[2, 0].plot(df1.Yt.diff().diff()); axes[2, 0].set_title('Diferenciação - 2a. Ordem')
plot_acf(df1.Yt.diff().diff().dropna(), ax=axes[2, 1])

plt.show()
```

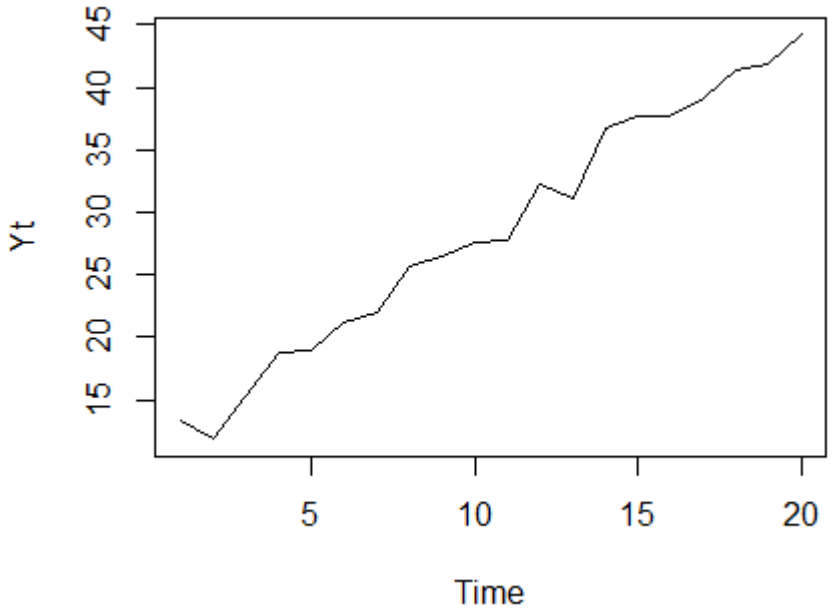
Série Estacionária



Série Estacionária

- Exemplo de *detrending* – considere a série histórica abaixo, cuja curva de regressão obtida foi $\widehat{Y}_t = 10 + 1{,}78t$

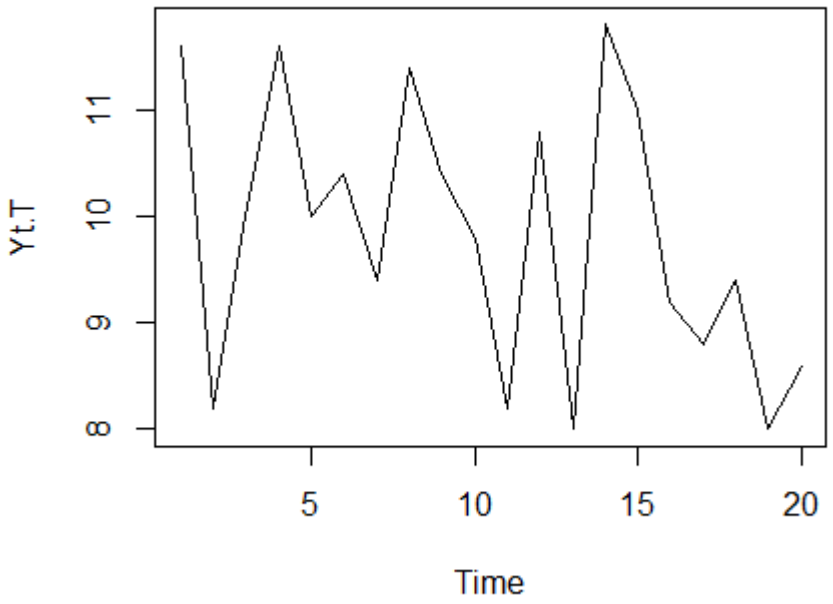
t	yt	t	yt
1	13,4	11	27,8
2	11,8	12	32,2
3	15,3	13	31,1
4	18,7	14	36,7
5	18,9	15	37,7
6	21,1	16	37,7
7	21,9	17	39,1
8	25,6	18	41,4
9	26,4	19	41,8
10	27,6	20	44,2



Série Estacionária

- Exemplo de *detrending* – removendo a componente de tendência $1{,}78t$, a nova série histórica é dada por:

t	yt	t	yt
1	11,6	11	8,2
2	8,2	12	10,8
3	10,0	13	8,0
4	11,6	14	11,8
5	10,0	15	11,0
6	10,4	16	9,2
7	9,4	17	8,8
8	11,4	18	9,4
9	10,4	19	8,0
10	9,8	20	8,6



- A nova série histórica é ~ estacionária.

Modelo Autoregressivo Integrado com Média Móvel ("ARIMA")

- Um modelo autoregressivo de ***p*** períodos de defasagem das variáveis 'independentes' Y_t, com ***d*** ajustes de diferenças, e ***q*** períodos de defasagem dos erros ϵ_t tem a seguinte estrutura:

$$\widehat{Y_t} = \theta_0 + \sum_{i=1}^{p} \phi_i Y_{t-i} + \sum_{i=1}^{q} w_i \epsilon_{t-i}$$

$$\epsilon_t = Y_t - \widehat{Y_t}$$

- "ARIMA" – *Autoregressive integrated moving average model*
- Usa-se a notação ARIMA(p,d,q).
- O uso de modelos ARIMA requer a determinação dos parâmetros ***p, d, q***.

ARIMA – Onde Entra o d

- No ARIMA(p,d,q), os termos AR e MA se aplicam à série **diferenciada d vezes**, W_t:
  - **d = 0:** $W_t = Y_t$ **d = 1:** $W_t = Y_t - Y_{t-1}$ **d = 2:** $W_t = (Y_t - Y_{t-1}) - (Y_{t-1} - Y_{t-2})$

$$\widehat{W_t} = \theta_0 + \sum_{i=1}^{p} \phi_i W_{t-i} + \sum_{i=1}^{q} w_i \epsilon_{t-i}$$

- A previsão em nível é obtida "integrando" de volta (d = 1): $\widehat{Y_t} = Y_{t-1} + \widehat{W_t}$; para prever h períodos futuros, $\widehat{Y}_{t+h} = Y_t + \widehat{W}_{t+1} + \cdots + \widehat{W}_{t+h}$
- Com d = 1, a constante é a variação média por período (*drift*). Exemplo 4, ARIMA(1,1,0): $\Delta\widehat{Y_t} = 0{,}4558 + 0{,}5522(\Delta Y_{t-1} - 0{,}4558)$.
- No Python basta informar a série original e *order=(p,d,q)*: a diferenciação e a reintegração são automáticas. Com d ≥ 1, a constante só entra com *trend='t'*.

Autocorrelação

- Autocorrelação é correlação de uma variável com ela mesma, com uma defasagem imposta de um número de períodos.
- A função de autocorrelação é a representação gráfica da autocorrelação para diferentes períodos de defasagem.
- A *autocorrelação parcial* para dada defasagem é a correlação entre as observações após remover o efeito das correlações devidas aos termos entre as observações.
- Na linguagem R: acf(*série_temporal*) e pacf(*série_temporal*)

Autocorrelação

	AR(p)	MA(q)	ARMA(p,q)
Função de Auto-correlação	Decai (~de forma exp.)	**Não-significativo após *q* períodos**	Decai (~de forma exp.)
Função de Auto-correlação Parcial	**Não-significativo após *p* períodos**	Decai (~de forma exp.)	Decai (~de forma exp.)

A *Função de Autocorrelação (ACF)* é a correlação de uma série temporal com ela mesma defasada de *p* períodos. A *Função de Autocorrelação Parcial (PACF)* é a correlação de uma série temporal com ela mesma defasada de *p* períodos depois de ter eliminado os efeitos de correlação para períodos menores. Portanto, PACF é a correlação específica para uma dada defasagem.

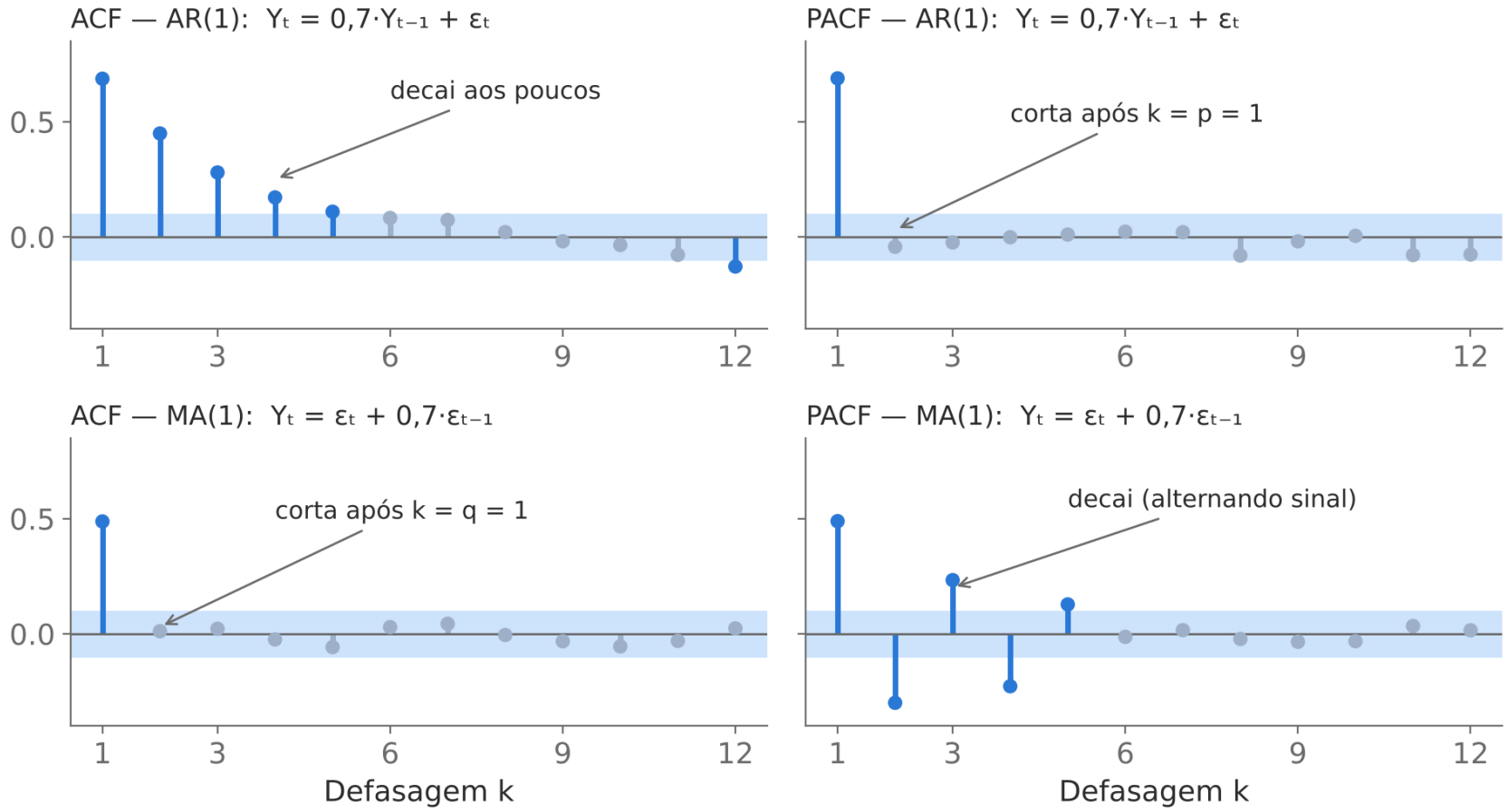
Como Ler a ACF e a PACF

- Eixo horizontal: defasagem k. Cada barra é a correlação entre Y_t e Y_{t-k} (ACF), ou essa correlação descontado o efeito das defasagens intermediárias (PACF). A defasagem 0 vale sempre 1 e é ignorada.
- Faixa sombreada: intervalo de 95% sob a hipótese de correlação nula, ≈ $\pm 1{,}96/\sqrt{n}$. Barra dentro da faixa → não significativa.
- "Corta" (*não-significativo após k*): a partir de uma defasagem, todas as barras caem dentro da faixa de uma vez. "Decai": as barras diminuem gradualmente, de forma exponencial ou oscilando.
- Leitura da tabela: **PACF corta em p** e ACF decai → AR(p); **ACF corta em q** e PACF decai → MA(q); ambas decaem → ARMA(p,q): testar p, q pequenos e comparar por AIC/BIC.
- Aplicar sempre à série **já estacionária** (após d diferenciações).

Por Que a ACF ou a PACF "Corta"?

- **AR(1):** Y_t depende diretamente só de Y_{t-1}, mas Y_{t-1} depende de Y_{t-2}, e assim por diante. A correlação com defasagens maiores é *indireta* e vai se diluindo: $ACF(k) = \varphi^k$, que decai. A PACF elimina os efeitos intermediários e mostra só a ligação direta: é nula para $k > p$.
- **MA(1):** $Y_t = \varepsilon_t + \theta\varepsilon_{t-1}$ e $Y_{t-2} = \varepsilon_{t-2} + \theta\varepsilon_{t-3}$ não compartilham nenhum choque ε, logo são não correlacionados: ACF nula para $k > q$. Já a PACF, ao descontar as defasagens intermediárias, não zera e decai.
- Na prática, com amostras finitas, uma barra isolada pode sair da faixa por acaso (em média 1 a cada 20 defasagens). Observe o **padrão**, não uma barra isolada; na dúvida, compare modelos candidatos por AIC/BIC.

ACF e PACF – Padrões Típicos



- Séries simuladas (n = 400). Barras em azul forte: fora da faixa de 95% (significativas); em cinza: dentro da faixa.

ARIMA (Exemplo 2)

- Considere a série histórica abaixo. Determine o número de períodos ***p*** a serem considerados no modelo autorregressivo.

t	yt	t	yt	t	yt	t	yt
1	5,0	16	7,0	31	4,6	46	5,1
2	4,0	17	5,4	32	4,5	47	4,8
3	6,9	18	5,0	33	4,0	48	2,6
4	8,0	19	2,8	34	3,3	49	5,5
5	8,3	20	2,6	35	4,4	50	4,0
6	8,0	21	4,8	36	3,3	51	5,2
7	6,5	22	4,8	37	5,7	52	5,0
8	7,3	23	7,7	38	5,9	53	6,4
9	6,6	24	5,4	39	5,9	54	7,9
10	6,3	25	4,8	40	8,0	55	6,1
11	4,0	26	4,2	41	5,3	56	5,0
12	5,4	27	5,8	42	7,1	57	3,6
13	3,0	28	5,5	43	6,9	58	5,0
14	6,5	29	6,4	44	5,3	59	3,3
15	6,7	30	4,2	45	7,0	60	5,4

ARIMA (Exemplo 2)

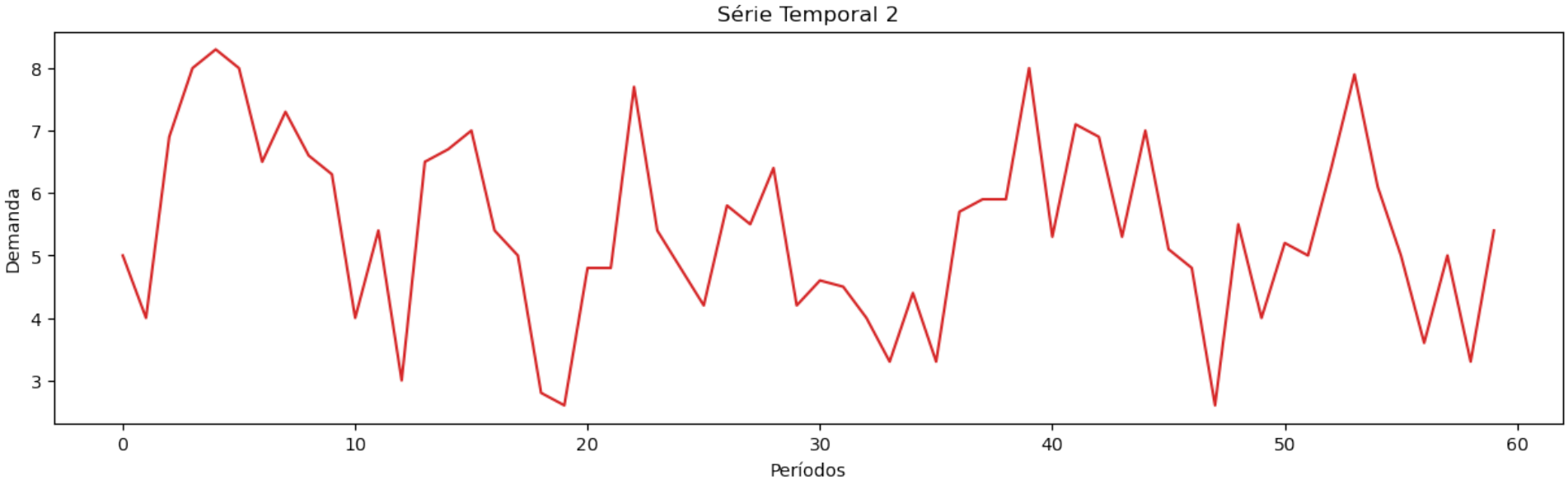
```
data2 = [5.0, 4.0, 6.9, 8.0, 8.3, 8.0, 6.5, 7.3, 6.6, 6.3, 4.0, 5.4, 3.0, 6.5, 6.7,
         7.0, 5.4, 5.0, 2.8, 2.6, 4.8, 4.8, 7.7, 5.4, 4.8, 4.2, 5.8, 5.5, 6.4, 4.2,
         4.6, 4.5, 4.0, 3.3, 4.4, 3.3, 5.7, 5.9, 5.9, 8.0, 5.3, 7.1, 6.9, 5.3, 7.0,
         5.1, 4.8, 2.6, 5.5, 4.0, 5.2, 5.0, 6.4, 7.9, 6.1, 5.0, 3.6, 5.0, 3.3, 5.4]
df2 = pd.DataFrame(
    data  = {'Yt': data2})
df2.head()
```

	Yt
0	5.0
1	4.0
2	6.9
3	8.0
4	8.3

ARIMA (Exemplo 2)

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15,4), dpi=100)
plt.plot(df2.index, df2.Yt, color='tab:red')
plt.gca().set(title='Série Temporal 2', xlabel='Períodos', ylabel='Demanda')
plt.show()
```



ARIMA (Exemplo 2)

```
from statsmodels.tsa.stattools import adfuller
from numpy import log

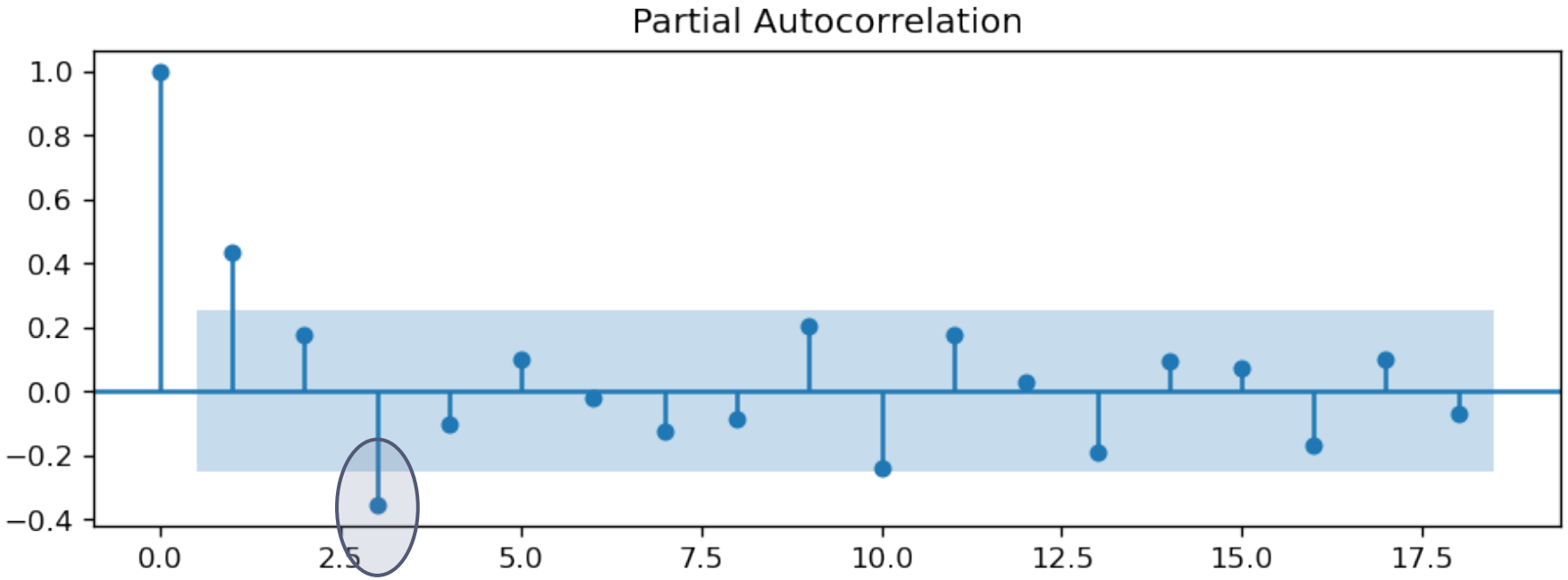
result = adfuller(df2.Yt.dropna())
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
```

```
ADF Statistic: -4.147215
p-value: 0.000808
```

Rejeitamos a hipótese nula, e a série é estacionária.

ARIMA (Exemplo 2)

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt
plt.rcParams.update({'figure.figsize':(9,3), 'figure.dpi':120})
plot_pacf(df2.Yt.dropna())
plt.show()
```



O modelo autorregressivo será de ordem 3.

ARIMA (Exemplo 2)

- Recorrendo ao método dos mínimos quadrados, o modelo autorregressivo resultante é um AR(3): $\widehat{Y_t} = 3 + 0{,}63Y_{t-1} + 0{,}27Y_{t-2} - 0{,}40Y_{t-3}$.

ARIMA (Exemplo 3)

- Considere a série histórica abaixo. Determine o número de períodos ***q*** a serem considerados no modelo de média móvel.

t	yt	t	yt	t	yt	t	yt
1	7,5	16	9,0	31	9,1	46	7,8
2	7,0	17	7,0	32	10,3	47	5,9
3	6,0	18	8,7	33	7,5	48	8,6
4	9,1	19	7,7	34	6,9	49	6,3
5	9,5	20	9,6	35	8,9	50	5,2
6	8,5	21	10,9	36	7,0	51	5,8
7	9,6	22	10,8	37	7,5	52	5,6
8	6,7	23	10,1	38	7,5	53	9,0
9	8,3	24	9,0	39	8,5	54	6,6
10	6,4	25	10,2	40	7,1	55	6,1
11	8,6	26	8,6	41	7,5	56	7,8
12	9,2	27	7,5	42	9,2	57	7,3
13	8,5	28	8,6	43	7,6	58	9,7
14	9,5	29	6,6	44	9,8	59	9,3
15	10,2	30	6,7	45	10,5	60	9,9

ARIMA (Exemplo 3)

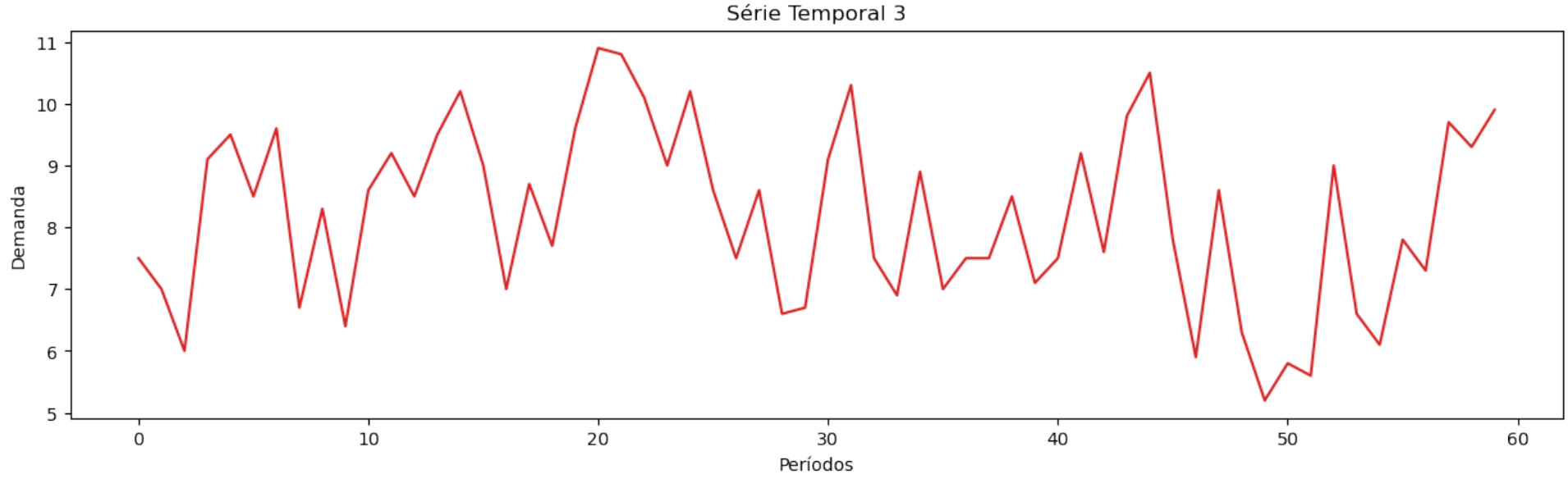
```
data3 = [ 7.5,  7.0, 6.0, 9.1, 9.5,  8.5,  9.6,  6.7, 8.3,  6.4, 8.6, 9.2, 8.5, 9.5, 10.2,
          9.0,  7.0, 8.7, 7.7, 9.6, 10.9, 10.8, 10.1, 9.0, 10.2, 8.6, 7.5, 8.6, 6.6,  6.7,
          9.1, 10.3, 7.5, 6.9, 8.9,  7.0,  7.5,  7.5, 8.5,  7.1, 7.5, 9.2, 7.6, 9.8, 10.5,
          7.8,  5.9, 8.6, 6.3, 5.2,  5.8,  5.6,  9.0, 6.6,  6.1, 7.8, 7.3, 9.7, 9.3,  9.9]
df3 = pd.DataFrame(
    data  = {'Yt': data3})
df3.head()
```

	Yt
0	7.5
1	7.0
2	6.0
3	9.1
4	9.5

ARIMA (Exemplo 3)

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15,4), dpi=100)
plt.plot(df3.index, df3.Yt, color='tab:red')
plt.gca().set(title='Série Temporal 3', xlabel='Períodos', ylabel='Demanda')
plt.show()
```



ARIMA (Exemplo 3)

```
from statsmodels.tsa.stattools import adfuller
from numpy import log

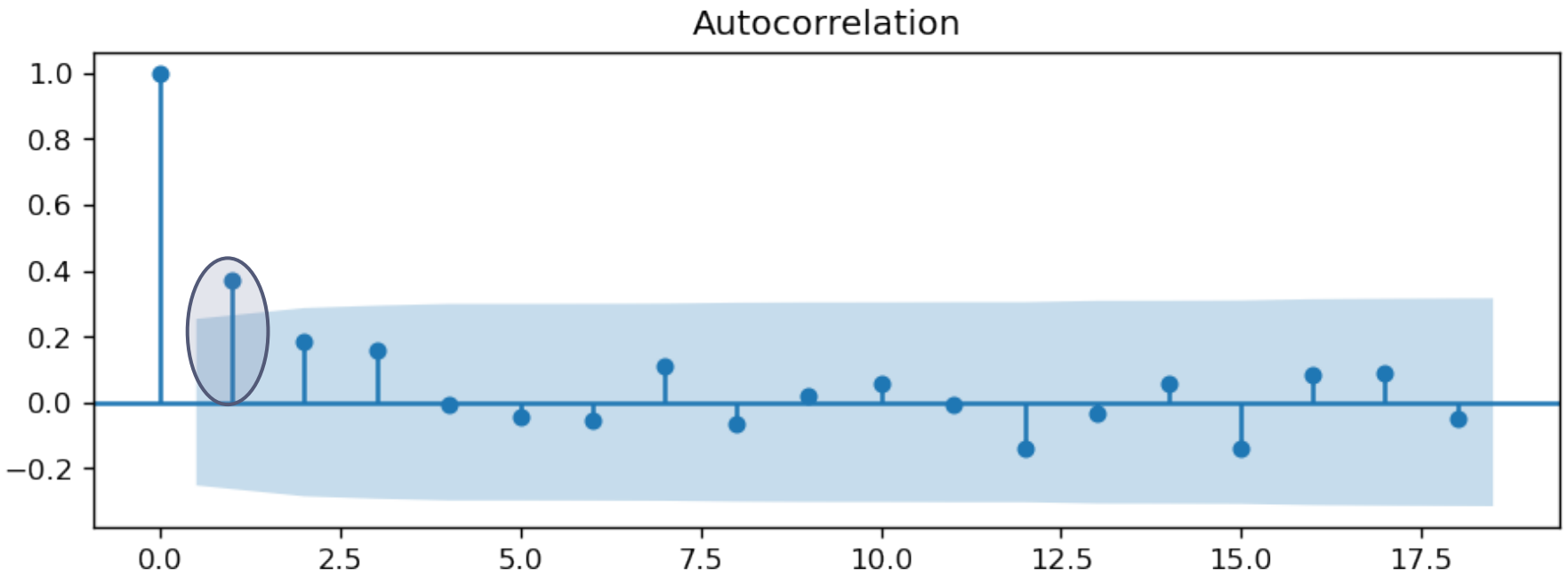
result = adfuller(df3.Yt.dropna())
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
```

```
ADF Statistic: -4.984166
p-value: 0.000024
```

Rejeitamos a hipótese nula, e a série é estacionária.

ARIMA (Exemplo 3)

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt
plt.rcParams.update({'figure.figsize':(9,3), 'figure.dpi':120})
plot_acf(df3.Yt.dropna())
plt.show()
```



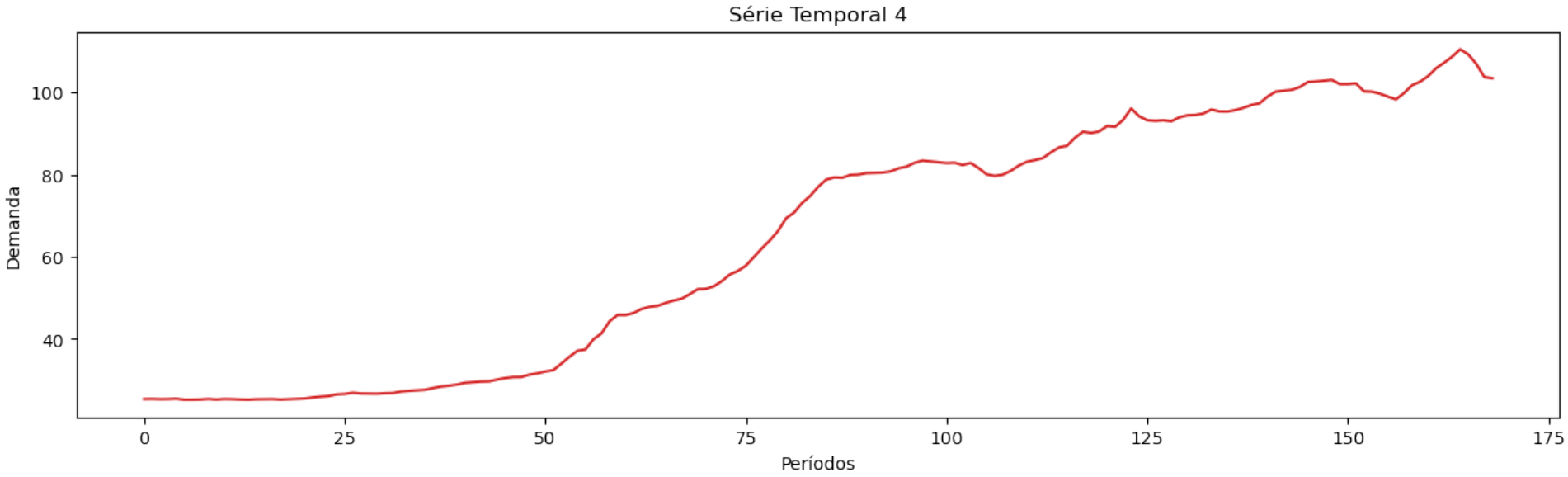
O modelo de média móvel será de ordem 1.

ARIMA (Exemplo 3)

- Recorrendo ao método dos mínimos quadrados, o modelo de média móvel resultante é um MA(1): $\widehat{Y_t} = 8{,}1957 + 0{,}3523\epsilon_{t-1}$.

ARIMA (Exemplo 4)

- Considere a série histórica abaixo. Determine o melhor modelo ARIMA.



ARIMA (Exemplo 4)

```
data4 = [25.4, 25.440001, 25.370001, 25.4, 25.49, 25.24, 25.24, 25.280001, 25.42, 25.280001,
         25.41, 25.370001, 25.280001, 25.24, 25.34, 25.360001, 25.389999, 25.27, 25.35, 25.42,
         25.530001, 25.790001, 25.98, 26.129999, 26.530001, 26.639999, 26.93, 26.75, 26.74,
         26.700001, 26.809999, 26.860001, 27.23, 27.4, 27.51, 27.65, 28.07, 28.440001, 28.65,
         28.93, 29.35, 29.49, 29.66, 29.700001, 30.139999, 30.5, 30.74, 30.76, 31.33, 31.639999,
         32.110001, 32.450001, 34.02, 35.669998, 37.139999, 37.459999, 39.939999, 41.360001,
         44.299999, 45.84, 45.830002, 46.32, 47.310001, 47.82 ,48.060001, 48.77, 49.360001,
         49.799999, 50.900002, 52.119999, 52.189999, 52.799999, 54.080002, 55.700001, 56.560001,
         57.84, 59.939999 ,62.07, 64.010002, 66.269997, 69.349998, 70.739998, 73.080002, 74.769997,
         77, 78.75, 79.279999, 79.199997, 79.870003, 79.959999, 80.32, 80.400002, 80.449997, 80.730003,
         81.5, 81.900002, 82.82, 83.370003, 83.169998, 82.959999, 82.769997, 82.849998, 82.290001,
         82.800003, 81.540001, 80.019997, 79.650002, 79.940002, 80.879997, 82.160004, 83.089996,
         83.489998, 83.980003, 85.370003, 86.57, 86.970001, 88.919998, 90.410004, 90.089996, 90.440002,
         91.779999, 91.589996, 93.269997, 96.050003, 94.129997, 93.190002, 93.029999, 93.169998, 92.93,
         93.889999, 94.370003, 94.449997, 94.82, 95.790001, 95.330002, 95.300003, 95.650002, 96.209999,
         96.910004, 97.309998, 98.940002, 100.14, 100.35, 100.57, 101.24, 102.47, 102.6, 102.81, 103.0,
         101.96, 101.96, 102.14, 100.22, 100.14, 99.629997, 98.910004, 98.269997, 99.790001, 101.69,
         102.55, 103.91, 105.83, 107.17, 108.64, 110.43, 109.2, 106.9, 103.69, 103.4]
df4 = pd.DataFrame(
    data  = {'Yt': data4})
df4.head()
```

	Yt
0	25.400000
1	25.440001
2	25.370001
3	25.400000
4	25.490000

ARIMA (Exemplo 4)

```
from statsmodels.tsa.stattools import adfuller
from numpy import log

result = adfuller(df4.Yt.dropna())
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
```

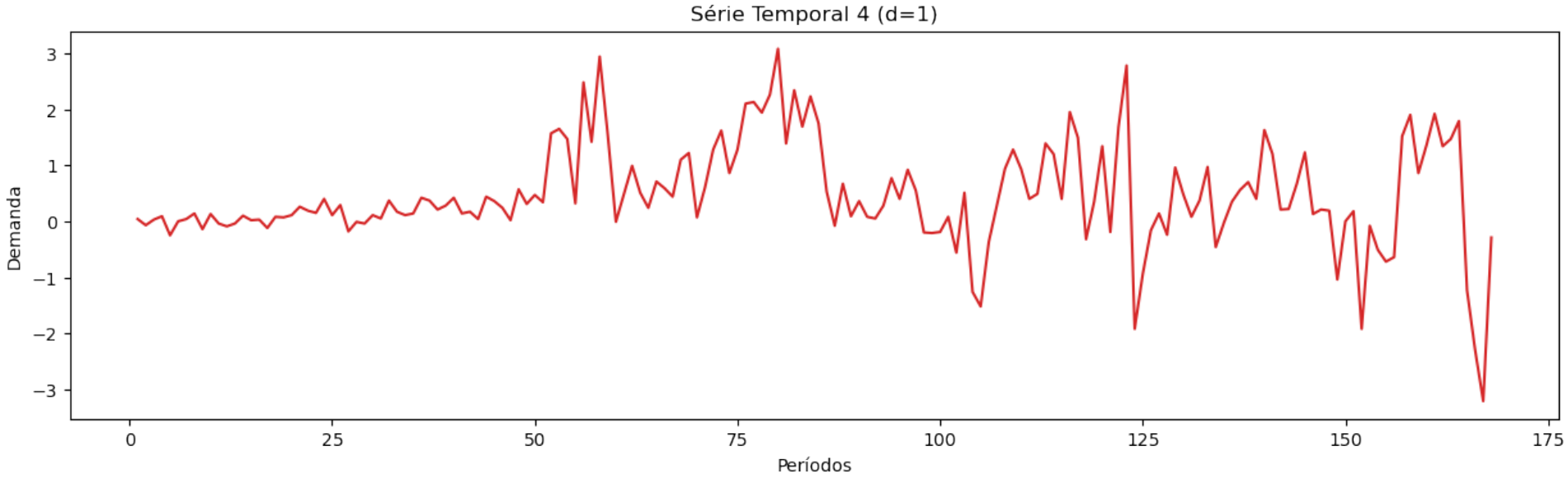
```
ADF Statistic: -0.774562
p-value: 0.826480
```

Aceitamos a hipótese nula, e a série é não-estacionária.

ARIMA (Exemplo 4) – Uma diferenciação

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15,4), dpi=100)
plt.plot(df4.Yt.diff().dropna(), color='tab:red')
plt.gca().set(title='Série Temporal 4 (d=1)', xlabel='Períodos', ylabel='Demanda')
plt.show()
```



ARIMA (Exemplo 4) – Uma diferenciação

```
from statsmodels.tsa.stattools import adfuller
from numpy import log

result = adfuller(df4.Yt.diff().dropna())
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
```

```
ADF Statistic: -4.047743
p-value: 0.001180
```

- Dado que a série é estacionária, determinar os coeficientes ***p*** e ***q,*** e o respectivo modelo ARIMA.
- Aplicação da Metodologia Box-Jenkins.

Metodologia Box-Jenkins

- A metodologia Box-Jenkins se refere ao processo iterativo de uso de modelos ARIMA, em que a consistência do modelo é verificada, com as checagens de série estacionária (Teste de Dickey-Fuller) e da consistência dos resíduos (Teste de Ljung-Box-Pierce).
- Em se identificando falhas no modelo, o processo é repetido variando-se os parâmetros *p*, *d* e *q*.

Biblioteca ARIMA – Versão Atual do statsmodels

```
from statsmodels.tsa.arima.model import ARIMA      # (antes: tsa.arima_model)

ordens = [(1,1,0), (0,1,1), (1,1,1), (2,1,0), (2,1,1), (2,1,2)]
for o in ordens:
    fit = ARIMA(df4.Yt, order=o, trend='t').fit()  # trend='t' = constante com d=1
    print(o, round(fit.aic, 2), round(fit.bic, 2))

fit = ARIMA(df4.Yt, order=(1,1,0), trend='t').fit()
print(fit.summary())                               # coeficientes: x1 (const.), ar.L1
```

Modelo	(1,1,0)	(0,1,1)	(1,1,1)	(2,1,0)	(2,1,1)	(2,1,2)
AIC	392,79	404,99	393,17	394,15	394,36	394,29
BIC	402,16	414,36	405,67	406,65	409,98	413,03

- Os resultados coincidem com a tabela do slide "Resumo dos Modelos ARIMA" (mesmos coeficientes, AIC e BIC).
- **Mudanças:** *fit()* sem *disp*; previsão com *get_forecast(h)* (média: *.predicted_mean*; IC: *.conf_int()*); a constante passa a se chamar *x1*.

Resumo dos Modelos ARIMA

	ARIMA (1,1,0)	ARIMA (0,1,1)	ARIMA (1,1,1)	ARIMA (2,1,0)	ARIMA (2,1,1)	ARIMA (2,1,2)
θ_0	0,4558	0,4654	0,4397	0,4509	0,4353	0,4326
ϕ_1	0,5522		0,7245	0,5173	0,9876	0,5080
ϕ_2				0,0650	-0,1628	0,2246
w_1		0,4872	-0,2547		-0,5019	0,0083
w_2						-0,2244
AIC	392,79	404,99	393,17	394,15	394,36	394,29
BIC	402,16	414,36	405,67	406,65	409,98	413,03

- Como identificar o melhor modelo? Pelos critérios AIC e BIC.

AIC e BIC – Critérios para comparação entre modelos

- O Critério de Informação de Akaike (AIC) é definido como:

$$AIC_p = -2\log(L_p) + 2[(p+1)+1]$$

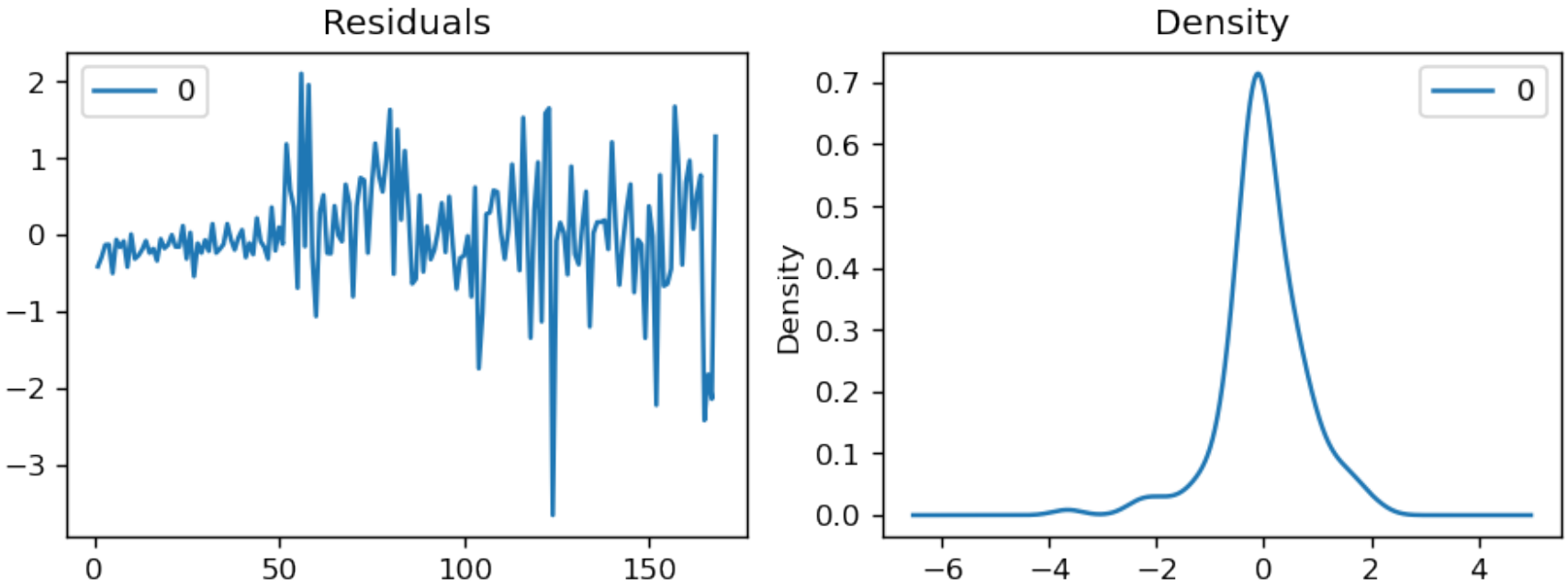
- onde: p – número de variáveis do modelo, L_p - função de máxima verossimilhança do modelo.
- O Critério de Informação Bayesiano (BIC) é definido como:

$$BIC_p = -2\log(L_p) + [(p+1)+1]\log(n)$$

- Tanto o AIC quanto o BIC aumentam conforme SQE aumenta, e ambos penalizam modelos com muitas variáveis. Assim, **valores menores de AIC e BIC são preferíveis**.

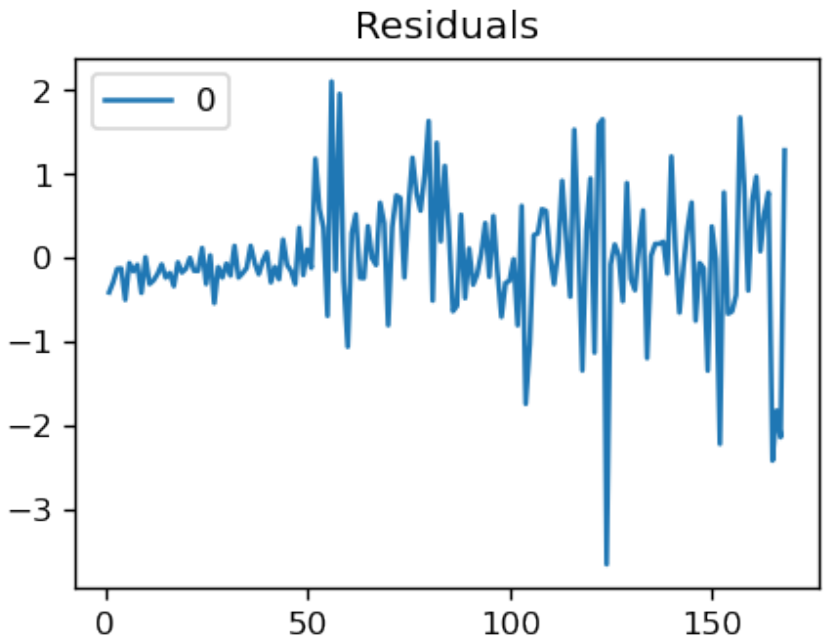
Resíduos do Modelo de Previsão

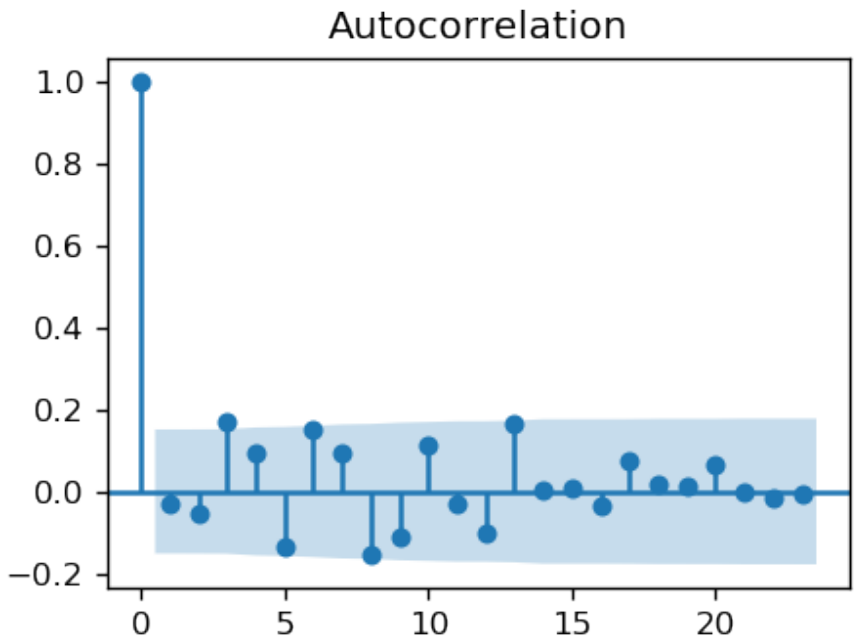
```
# Plot residual errors
residuals = pd.DataFrame(model_fit.resid)
fig, ax = plt.subplots(1,2)
residuals.plot(title="Residuals", ax=ax[0])
residuals.plot(kind='kde', title='Density', ax=ax[1])
plt.show()
```



Resíduos do Modelo de Previsão

```
# Plot residual errors
residuals = pd.DataFrame(model_fit.resid)
fig, ax = plt.subplots(1,2)
residuals.plot(title="Residuals", ax=ax[0])
plot_acf(residuals, ax=ax[1])
plt.show()
```





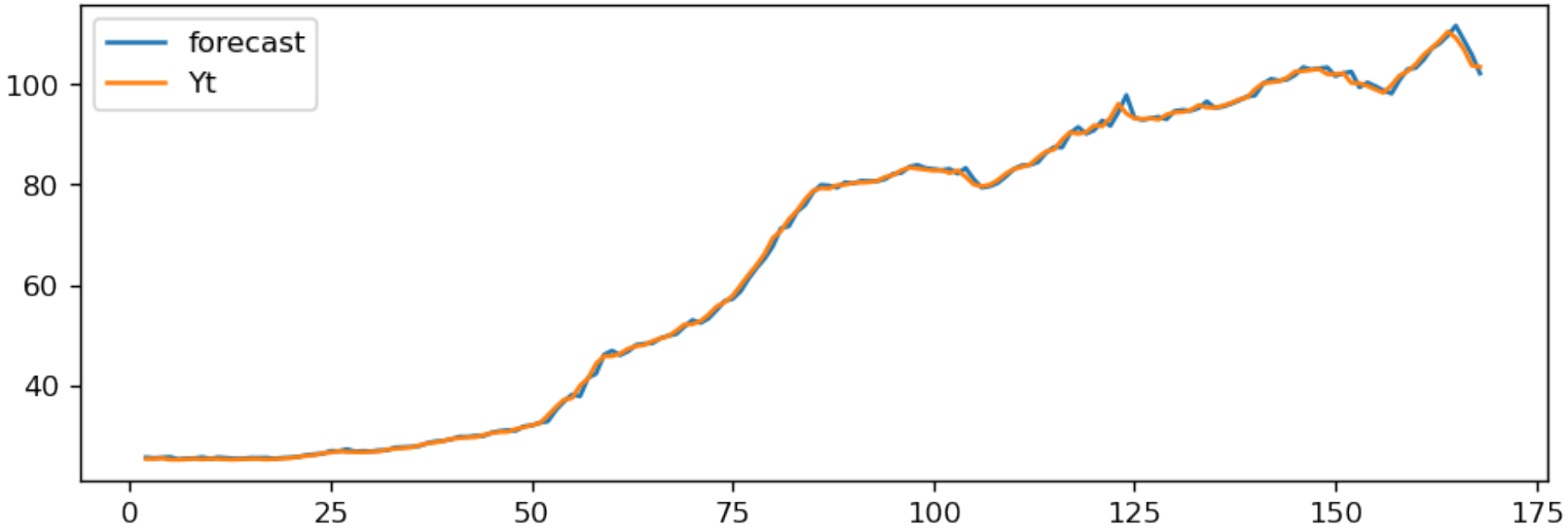
Resíduos do Modelo de Previsão

```
from statsmodels.graphics.tsaplots import acf
acf_res = acf(residuals)
acf_res
```

```
array([ 1.        , -0.02915076, -0.0539344 ,  0.16936136,  0.09502407,
       -0.13593078,  0.15257752,  0.09414418, -0.15236832, -0.10893412,
        0.11491307, -0.03179092, -0.10170875,  0.16304283,  0.00534045,
        0.00738156, -0.03530217,  0.07485032,  0.01799719,  0.01318307,
        0.06631255, -0.00174356, -0.01374802, -0.00482486,  0.10940925,
       -0.0414291 ,  0.00719089, -0.06143104,  0.15821108, -0.11039514,
       -0.0192513 ,  0.08015909,  0.06679565, -0.14908262,  0.02524686,
        0.05333201, -0.1379471 ,  0.03374102,  0.07380074, -0.1062046 ,
       -0.04714371])
```

Resíduos do Modelo de Previsão

```
# Dados reais vs Previsão
model_fit.plot_predict(dynamic=False)
plt.show()
```



Teste de Ljung-Box-Pierce

- A adequação do modelo de previsão pode ser verificada pelo teste Ljung-Box-Pierce, que analisa a consistência dos resíduos, fazendo uso da autocorrelação dos mesmos, e aplicando o teste do χ^2.

$$Q_m = n(n+2)\sum_{k=1}^{m}\frac{{r_k}^2}{n-k}$$

- onde: n – número de observações da série; k – defasagem; m – número máximo de defasagens; r_k - autocorrelação (amostral) dos resíduos com defasagem de k períodos.
- Se $Q_m > \chi^2(\alpha, m-p-q)$ rejeitar o modelo.

Teste de Ljung-Box-Pierce

m	ACF(k)	ACF(k)^2	ACF(k)^2 / (n-m)	Ac	Qm	$\chi^2(1\%, m-1)$
1	-0,029151	0,0008498	5,05814E-06	5,05814E-06	0,1461751	--
2	-0,053934	0,0029089	1,74187E-05	2,24768E-05	0,6495575	6,634896601
3	0,1693614	0,0286833	0,000172791	0,000195268	5,6430384	9,210340372
4	0,0950241	0,0090296	5,47247E-05	0,000249992	7,2245272	11,34486673
5	-0,135931	0,0184772	0,000112666	0,000362658	10,480454	13,27670414
6	0,1525775	0,0232799	0,000142821	0,000505479	14,607851	15,08627247
7	0,0941442	0,0088631	5,47107E-05	0,00056019	16,188935	16,81189383
8	-0,152368	0,0232161	0,000144199	0,00070439	20,356153	18,47530691
9	-0,108934	0,0118666	7,41665E-05	0,000778556	22,499491	20,09023503
10	0,1149131	0,013205	8,30504E-05	0,000861606	24,899565	21,66599433
11	-0,031791	0,0010107	6,3966E-06	0,000868003	25,08442	23,20925116
12	-0,101709	0,0103447	6,58896E-05	0,000933893	26,988564	24,72497031
13	0,1630428	0,026583	0,000170404	0,001104296	31,913059	26,21696731
14	0,0053405	2,852E-05	1,84003E-07	0,00110448	31,918376	27,68824961
15	0,0073816	5,449E-05	3,53814E-07	0,001104834	31,928601	29,14123774
16	-0,035302	0,0012462	8,14538E-06	0,001112979	32,163994	30,57791417
17	0,0748503	0,0056026	3,6859E-05	0,001149839	33,229183	31,99992691
18	0,0179972	0,0003239	2,14503E-06	0,001151984	33,291172	33,40866361
19	0,0131831	0,0001738	1,15862E-06	0,001153142	33,324655	34,80530573
20	0,0663126	0,0043974	2,95124E-05	0,001182655	34,177535	36,19086913

Teste de Ljung-Box-Pierce no Python

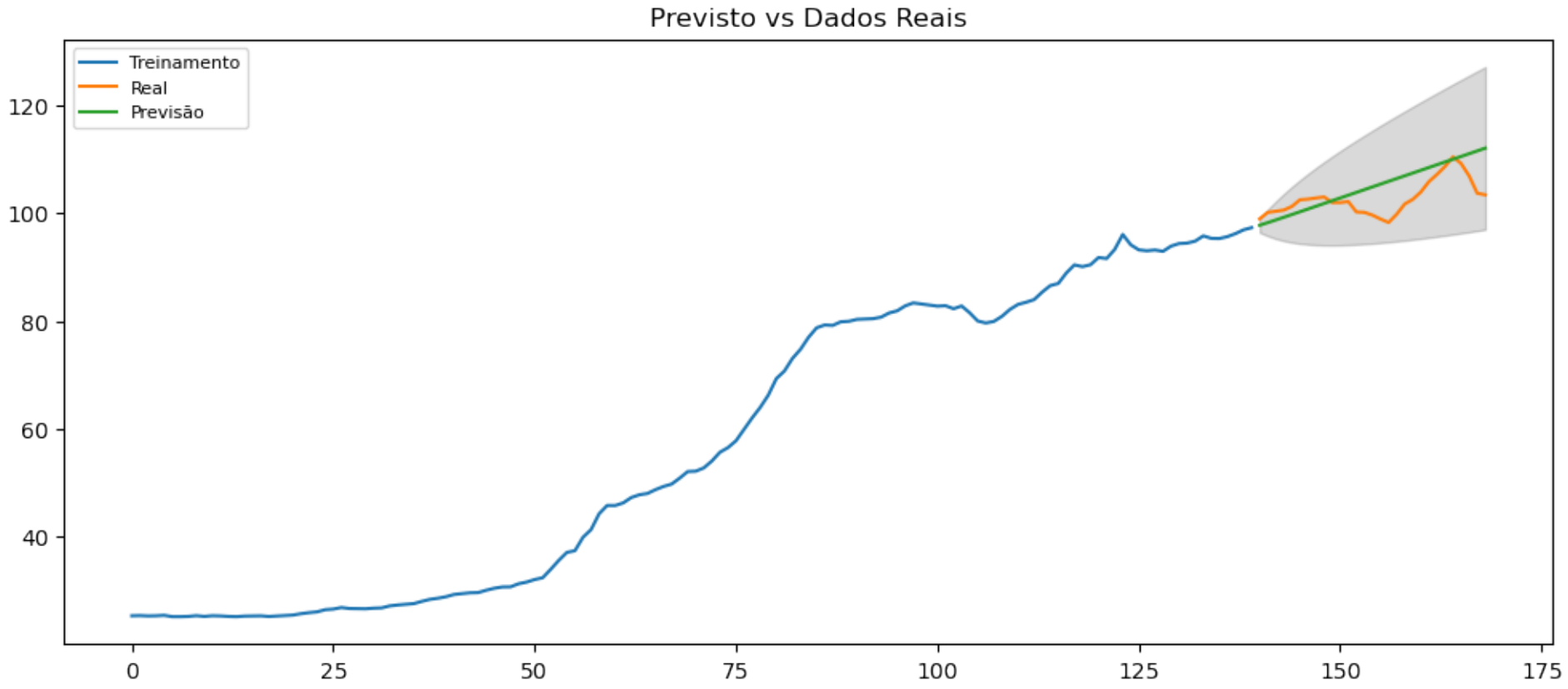
```
from statsmodels.stats.diagnostic import acorr_ljungbox

fit = ARIMA(df4.Yt, order=(1,1,0), trend='t').fit()
res = fit.resid[1:]          # descarta o 1º resíduo (artefato da diferenciação)
acorr_ljungbox(res, lags=[5, 10, 15, 20], model_df=1)    # model_df = p + q
```

m	5	10	15	20
Qm	10,42	24,76	31,75	33,99
p-valor	0,034	0,003	0,004	0,018

- Hipótese nula: resíduos sem autocorrelação. **p-valor < α → rejeitar o modelo.**
- A conclusão depende de m e de α : a 1% (critério do slide anterior) o modelo passa em m = 5 e 20, mas não em m = 10 e 15; a 5%, é rejeitado em todos.
- Os resíduos têm variância crescente (ver gráfico de resíduos). Uma alternativa é modelar log(Yt): o ARIMA(1,1,1) em log passa no teste (p = 0,07 em m = 10; 0,33 em m = 20).

Previsão Baseada no Modelo ARIMA



Como Usar o ARIMA – Roteiro

1. Visualizar a série: tendência, sazonalidade, valores atípicos.
2. Separar **treino e teste** (cronologicamente).
3. Estacionariedade no treino: teste de Dickey-Fuller; diferenciar até estacionar $\longrightarrow d$.
4. Identificação: ACF e PACF da série diferenciada $\longrightarrow$ candidatos p e q.
5. Estimação: ajustar os candidatos e comparar AIC/BIC.
6. Diagnóstico: resíduos sem autocorrelação (ACF, Ljung-Box-Pierce) e aproximadamente normais.
7. Validação: prever o período de teste, medir o erro (RMSE, MAE, MAPE) e comparar com um modelo ingênuo (ou um mais simples, como o suavizamento exponencial).
8. Reajustar o modelo escolhido com a série completa e prever o futuro com intervalo de confiança.

Treino e Teste em Séries Temporais

- AIC/BIC e resíduos medem o ajuste ao passado. Só o período de teste mostra se o modelo prevê bem dados que não viu.
- Divisão **cronológica**: treino = início da série; teste = final (tipicamente 15–25%, ou o horizonte de previsão desejado). Nunca embaralhar: o modelo estaria usando o futuro para prever o passado.
- Duas formas de avaliar: 1) **Previsão de h passos**: prevê todo o teste de uma vez (como prever 29 períodos à frente). 2) **Previsão rolante de 1 passo** (walk-forward): a cada período incorpora o valor real e prevê o seguinte.
- Métricas: $RMSE = \sqrt{\frac{\sum e_t^2}{n}}, MAE = \frac{\sum |e_t|}{n}, MAPE = 100\frac{\sum \left|\frac{e_t}{Y_t}\right|}{n}$
- Referência mínima: o modelo **ingênuo** ($\hat{Y}_{t+1} = Y_t$). Um ARIMA que não o supera não se justifica.

Treino e Teste – Código (Exemplo 4)

```
from statsmodels.tsa.arima.model import ARIMA
import numpy as np

train, test = df4.Yt[:140], df4.Yt[140:]

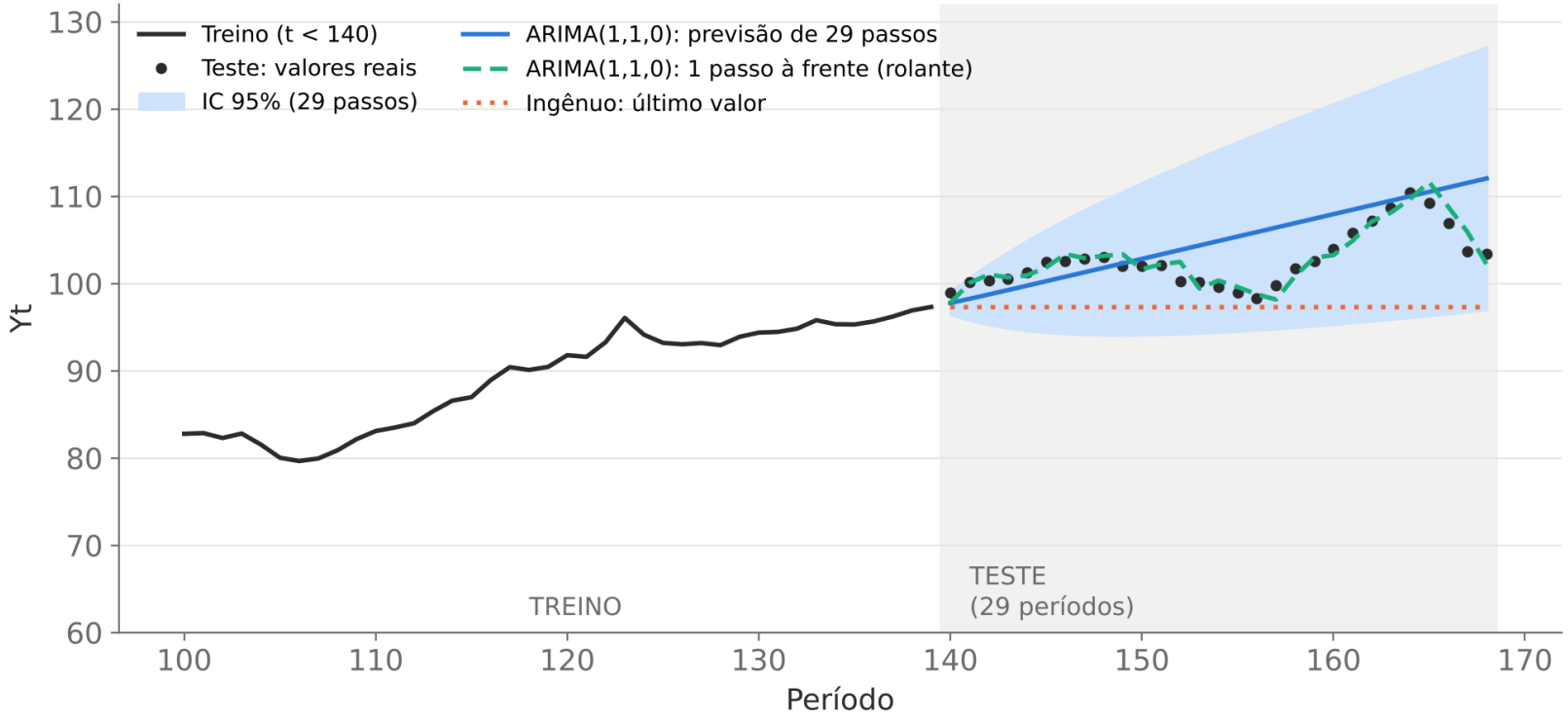
# Ajuste no treino (trend='t': constante da série diferenciada)
model = ARIMA(train, order=(1,1,0), trend='t').fit()

# a) Previsão de 29 passos, com IC 95%
prev = model.get_forecast(steps=len(test))
fc, ic = prev.predicted_mean, prev.conf_int(alpha=0.05)

# b) Previsão rolante de 1 passo (parâmetros mantidos)
res, one = model, []
for y in test:
    one.append(res.forecast(1).iloc[0])
    res = res.append([y])

def metricas(y, f):          # RMSE, MAE, MAPE
    e = np.asarray(y) - np.asarray(f)
    return np.sqrt(np.mean(e**2)), np.mean(abs(e)), 100*np.mean(abs(e/np.asarray(y)))
print(metricas(test, fc), metricas(test, one))
```

Treino e Teste – Resultado (Exemplo 4)



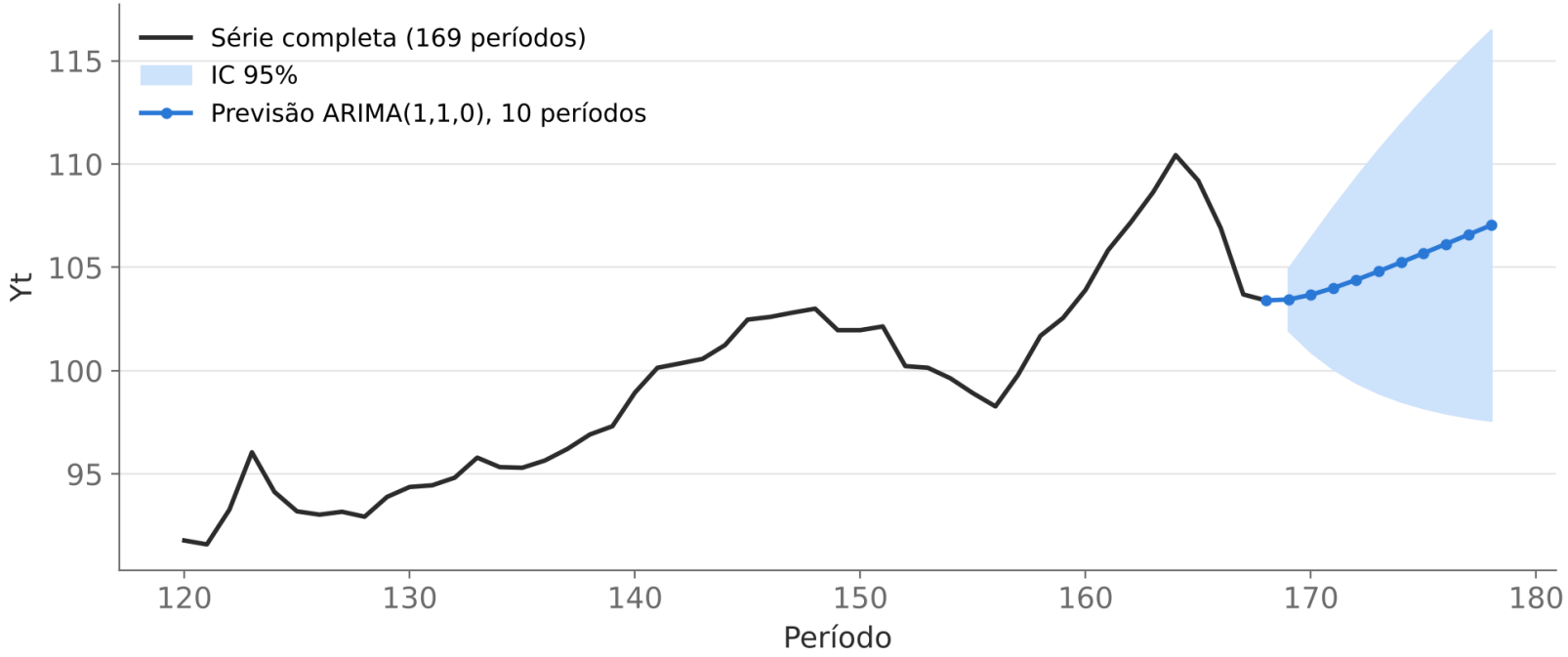
- A previsão de 29 passos segue só a tendência média; a previsão rolante de 1 passo acompanha as oscilações porque recebe os valores reais a cada período.

Treino e Teste – Comparação dos Modelos

Modelo	AIC (treino)	RMSE 29 passos	RMSE 1 passo
ARIMA(1,1,0)	**295,2**	**3,99**	**1,07**
ARIMA(0,1,1)	307,0	4,06	1,10
ARIMA(1,1,1)	288,6	3,77	1,15
ARIMA(2,1,0)	294,5	3,97	1,09
ARIMA(2,1,1)	290,0	3,66	1,14
ARIMA(2,1,2)	289,9	3,63	1,13
Ingênuo ($\hat{Y}_{t+1} = Y_t$)	—	6,25	1,30

- Todos os ARIMA superam o ingênuo, sobretudo no horizonte longo (RMSE 3,6–4,1 contra 6,3).
- O erro cresce muito com o horizonte (≈ 1,1 em 1 passo; ≈ 3,6–4,0 em 29 passos); por isso o intervalo de confiança se abre.
- As diferenças entre os ARIMA são pequenas, e o menor AIC não garante o menor erro no teste. Entre modelos equivalentes, prefira o mais simples (parcimônia): aqui, **ARIMA(1,1,0)**.

Previsão Final



- Escolhido o modelo, ele é reajustado com toda a série (169 períodos): o teste serve para escolher o modelo, não para descartar dados.
- Previsão: 103,44 no 1º período e 107,04 no 10º; IC 95% no 10º período: [97,6 ; 116,5].

Exercícios

Exercício 1

- Para os dados indicados na tabela, representativos de um processo de controle de qualidade, identifique um modelo ARIMA(*p,d,q*) válido, e faça a previsão para as próximas 7 medições.

-0,23	-0,20	-1,93	-0,97	0,10
0,63	-0,21	1,87	0,83	-0,62
0,48	0,91	-0,97	-0,33	2,27
-0,83	-0,36	0,46	0,91	-0,62
-0,03	0,48	2,12	-1,13	0,74
1,31	0,61	-2,11	2,22	-0,16
0,86	-1,38	0,70	0,80	1,34
-1,28	-0,04	0,69	-1,95	-1,83
0,00	0,90	-0,24	2,61	0,31
-0,63	1,79	0,34	0,59	1,13
0,08	-0,37	0,60	0,71	-0,87
-1,30	0,40	0,15	-0,84	1,45
1,48	-1,19	-0,02	-0,11	-1,95
-0,28	0,98	0,46	1,27	-0,51
-0,79	-1,51	-0,54	-0,80	-0,41
1,86	0,90	0,89	-0,76	0,49
0,07	-1,56	1,07	1,58	1,54
0,09	2,18	0,20	-0,38	-0,96

Exercício 2

- Para os dados mostrados na tabela, indicativos do valor de fechamento das ações de uma empresa de tecnologia nos últimos 65 dias, faça a previsão para os próximos 5 dias usando um modelo ARIMA.

222,34	233,05	253,41
222,24	235,00	252,04
221,17	236,17	248,78
218,88	238,31	247,76
220,05	241,14	249,27
219,61	241,48	247,95
216,40	246,74	251,41
217,33	248,73	254,67
219,69	248,83	258,62
219,32	248,78	259,25
218,25	249,61	261,49
220,30	249,90	264,95
222,54	246,65	268,21
223,56	247,57	272,16
223,07	247,76	272,79
225,36	247,81	275,03
227,60	250,68	278,49
226,82	251,80	281,75
229,69	251,07	285,70
229,30	248,05	286,33
228,96	249,76	288,57
229,99	251,66	